EML — hybrid formal/informal report

Auto-formalization of arXiv:2603.21852 (Odrzywołek)

2026-05-02

Contents

EML formalization — hybrid report

This document interleaves the paper *All elementary functions from a single binary operator* (arXiv:2603.21852, A. Odrzywołek) with the corresponding Lean 4 + Mathlib v4.28 artifacts. Proofs were produced by Aristotle (Harmonic) plus hand-curated definitions.

Status dashboard

Status	Count	Symbol
Verified	45	
Partial	0	
Submitted	0	…
Failed	0	
Pending	0	·
Total	**45**	

Index

ID	Title	Kind	Diff	Section
001_def_eml	Definition of the EML operator	definition	1	§3 Results, Equation 3
002_def_eml_tree	Inductive type of EML terms	definition	1	§4.2 Elementary functions as binary trees
003_def_eml_eval	Evaluation of EML terms	definition	1	§4.1 EML compiler

	ID	Title	Kind	Diff	Section
	004_def_edl	EDL variant (Exp Divided by Log)	definition	1	§3 Results, Identity 4b
	005_def_neg_eml	Negated-EML variant	definition	1	§3 Results, Identity 4c
	006_eml_one_one	eml(1,1) = e	identity	1	§3 Results, EML expression catalog
	007_eml_x_one	eml(x,1) = exp(x)	identity	1	§3 Results, EML expression catalog
	008_eml_one_y	eml(1,y) = e − ln(y)	identity	1	§3 Results (consequence of Equation 3)
	009_eml_x_e	eml(x, e) = exp(x) − 1	identity	1	§3 Results (consequence of Equation 3)
	010_eml_pos_left	Positivity of the left exponential of eml	theorem	1	§3 Results (implicit; precondition lemma)
	011_ln_via_eml	Natural logarithm via EML	identity	3	§3 Results, Identity 5
	012_exp_via_eml	exp(x) as eml — corollary phrasing of 007	identity	2	§3 Results, EML expression catalog
	013_sub_via_eml	Subtraction expressed via EML	identity	2	§3 Results, Calculator-equivalence chain
	014_add_via_eml	Addition via Identity 1 (Exp-Log reduction)	identity	2	§3 Results, Identity 1

ID	Title	Kind	Diff	Section
015_mul_via_exp_log	Multiplication via Identity 1 (Exp-Log reduction)	identity	2	§3 Results, Identity 1
016_add_via_exp_log	Addition via Identity 1 (specialized)	identity	2	§3 Results, Identity 1 (specialised consequence)
017_successor_negation	Successor / negation identity	identity	2	§3 Results (passing remark)
018_inv_successor_algebraic	Algebraic simplification of inv(suc(inv x))	identity	1	§3 Results (sub-step of successor identity)
019_negation_calc3	Negation realised in the Calc-3 set	calculator-equivalence	3	§3 Results, Table 2 row ‘Calc 3’
020_emlterm_size	Size function on EML terms	definition	2	§4.1 EML compiler (‘K denotes the size of the RPN code’)
021_emlterm_size_pos	EML term size is positive	theorem	2	§4.1 EML compiler (implicit)
022_emlterm_witness	A witness EML term whose eval is e	theorem	2	§3 Results, EML expression catalog (e, K=3)
023_emlterm_exp_witness	EML term with x-leaf whose eval is exp(x)	theorem	3	§3 Results, EML expression catalog (exp(x), K=3)

ID	Title	Kind	Diff	Section
024_wolfram_to_calc3R	Wolfram NC → Calc3R (constant-free real subset)	calculator-equivalence	4	§3 Results, Table 2 (rows ‘Wolfram’ and ‘Calc 3’)
025_calc3_to_calc2	Calc 3 → Calc 2 reduction	calculator-equivalence	3	§3 Results, Table 2 (rows ‘Calc 3’ and ‘Calc 2’)
026_calc2_to_calc1	Calc 2 → Calc 1 reduction	calculator-equivalence	3	§3 Results, Table 2 (rows ‘Calc 2’ and ‘Calc 1’)
027_calc1_to_calc0	Calc 1 → Calc 0 reduction	calculator-equivalence	3	§3 Results, Table 2 (rows ‘Calc 1’ and ‘Calc 0’)
028_calc0_to_eml	Calc 0 → EML reduction	calculator-equivalence	4	§3 Results, Table 2 (rows ‘Calc 0’ and ‘EML’)
029_eml_minimality	Minimality: three primitives is the minimum	theorem	5	§3 Results (concluding remark on Table 2)
030_emlterm_for_zero	EML term whose eval is 0	theorem	4	§3 Results, EML expression catalog (0, K=7)
031_emlterm_for_neg_one	EML term whose eval is −1	theorem	4	§3 Results, EML expression catalog (−1, K=17)
032_emlterm_for_two	EML term whose eval is 2	theorem	4	§3 Results, EML expression catalog (2, K=27)

ID	Title	Kind	Diff	Section
033_emlterm_for_half	EML Term for 1/2	theorem	4	§3 Results, EML expression catalog (1/2, K=91)
034_emlterm_for_pi	EML Term for π	theorem	5	§3 Results, EML expression catalog (, K=193); Table S2 step 18
035_emlterm_for_i	EML Term for i	theorem	5	§3 Results, EML expression catalog (i, K=131); §2.1 compiler macros
036_emlterm_for_neg_x	EML Term for −x	theorem	5	§3 Results, EML expression catalog (−x, K=57)
037_emlterm_for_inv_x	EML Term for 1/x	theorem	5	§3 Results, EML expression catalog (1/x, K=65)
038_emlterm_for_sq_x	EML Term for x^2	theorem	5	§3 Results, EML expression catalog (x^2, K=75)
039_emlterm_for_sqrt_x	EML Term for √x	theorem	5	§3 Results, EML expression catalog (√x, K=139)
040_emlterm_for_add_xy	EML Term for x + y	theorem	5	§3 Results, EML expression catalog (x + y, K=27)

ID	Title	Kind	Diff	Section
041_emlterm_for_mul_xy	EMLTerm realising x · y	theorem	5	§3 Results, EML expression catalog (x × y, K=41)
042_emlterm_for_pow_xy	EMLTerm realising x^y (for 0 < x and 0 < y)	theorem	5	§3 Results, EML expression catalog (x^y, K=49)
043_master_formula_param_count	Master formula parameter count at level n	definition	2	§4.3 Master formula — symbolic regression
044_emlterm_count_catalan	Count of EMLTerms equals the Catalan number	theorem	4	§4.2 Elementary functions as binary trees ('Catalan structures')
045_main_completeness_stub	Main completeness theorem — eleven-conjunct umbrella	theorem	5	§3 Results, abstract claim of universality

001_def_eml Definition of the EML operator

Paper section: §3 Results, Equation 3 • *Status:* complete • *Difficulty:* 1/5

$$\mathrm{eml}(x, y) = \exp(x) - \ln(y)$$

The EML operator is a binary real-valued operator defined by eml(x,y) = exp(x) − ln(y). It is the heart of Odrzywołek's construction: combined with the constant 1 it generates every elementary function of a scientific calculator.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

/-- The EML (Exp-Minus-Log) binary operator on the reals.
Equation 3 in Odrzywołek (arXiv:2603.21852). -/
def eml (x y : ℝ) : ℝ := Real.exp x - Real.log y

end EML
```

002_def_eml_term Inductive type of EML terms

Paper section: §4.2 Elementary functions as binary trees • *Status:* complete • *Difficulty:* 1/5

S → 1 | eml(S, S)

EML terms are full binary trees: every leaf is the constant 1, and every internal node is an application of the eml operator to two subtrees. The language is isomorphic to the Catalan structures.

```
namespace EML

/-- Constant-only EML term grammar from §4.2:
    `S → 1 | eml(S, S)`. -/
inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

end EML
```

003_def_eml_eval Evaluation of EML terms

Paper section: §4.1 EML compiler • *Status:* complete • *Difficulty:* 1/5

Each EML term evaluates to a real number obtained by replacing every leaf 1 by the constant 1 and every internal node eml(t,u) by exp(eval t) − ln(eval u).

The eval function maps each EML term to a real number: a .one leaf gives 1, and a .eml t u node gives exp(eval t) − ln(eval u). Because Real.log is junk-valued at non-positive arguments the function is total on , but the 'meaningful' values arise only when the subtrees evaluate to positive reals.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

/-- Real-valued evaluation of an EML term. -/
def EMLTerm.eval : EMLTerm →
  | .one => 1
  | .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

end EML
```

004_def_edl EDL variant (Exp Divided by Log)

Paper section: §3 Results, Identity 4b • *Status:* complete • *Difficulty:* 1/5

edl(x, y) = exp(x) / ln(y), constant: e

The EDL variant uses division instead of subtraction: edl(x,y) = exp(x)/ln(y), with the constant e replacing 1. It has analogous universality properties but a different 'starting point'.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

/-- The EDL (Exp Divided by Log) variant of EML.
Identity 4b in Odrzywołek (arXiv:2603.21852). Constant: `e`. -/
def edl (x y : ) :   := Real.exp x / Real.log y

end EML
```

005_def_neg_eml Negated-EML variant

Paper section: §3 Results, Identity 4c • *Status:* complete • *Difficulty:* 1/5

$-\mathrm{eml}(y, x) = \ln(x) - \exp(y)$, constant: $-\infty$

The third variant negates EML and swaps the arguments: -eml(y,x) = ln(x) - exp(y). The paper labels its required constant as $-\infty$, reflecting a topological difference when trying to express 1.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

/-- The negated-EML variant: `-eml(y, x) = ln(x) - exp(y)`.
Identity 4c in Odrzywołek (arXiv:2603.21852). -/
def negEml (x y : ) :   := Real.log x - Real.exp y

end EML
```

006_eml_one_one_eq_e eml(1,1) = e

Paper section: §3 Results, EML expression catalog • *Status:* complete • *Difficulty:* 1/5

$e = \mathrm{eml}(1, 1)$

Applying the EML operator to two unit arguments yields Euler's number: eml(1,1) = exp(1) − ln(1) = e − 0 = e. The simplest of the fundamental examples.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

noncomputable def eml (x y : ) :   := Real.exp x - Real.log y

theorem eml_one_one : eml 1 1 = Real.exp 1 := by
  -- By definition of eml, we have eml 1 1 = Real.exp 1 - Real.log 1.
  simp [eml]

end EML
```

007_eml_x_one_eq_exp eml(x,1) = exp(x)

Paper section: §3 Results, EML expression catalog • *Status:* complete • *Difficulty:* 1/5

exp(x) = eml(x, 1)

Setting the second argument to 1 collapses ln(1) to zero, so eml(x,1) = exp(x) for every real x. This shows that the exponential is immediately reachable in EML.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

noncomputable def eml (x y : ) :  := Real.exp x - Real.log y

theorem eml_x_one (x : ) : eml x 1 = Real.exp x := by
  unfold eml; norm_num;

end EML
```

008_eml_one_y eml(1,y) = e − ln(y)

Paper section: §3 Results (consequence of Equation 3) • *Status:* complete • *Difficulty:* 1/5

eml(1, y) = exp(1) − ln(y)

Setting the first argument to 1 reduces exp(1) to the constant e, giving eml(1,y) = e − ln(y). The hypothesis y > 0 is not formally required (Real.log is junk-valued elsewhere) but we keep it for semantic clarity.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

noncomputable def eml (x y : ) :  := Real.exp x - Real.log y

theorem eml_one_y (y : ) (hy : 0 < y) : eml 1 y = Real.exp 1 - Real.log y := by
  simp [eml]

end EML
```

009_eml_x_e eml(x, e) = exp(x) − 1

Paper section: §3 Results (consequence of Equation 3) • *Status:* complete • *Difficulty:* 1/5

eml(x, e) = exp(x) − ln(e) = exp(x) − 1

Substituting y = e yields ln(e) = 1, so eml(x,e) = exp(x) − 1. The value exp(x) − 1 appears throughout analysis (Taylor expansions, etc.) so it is worth isolating.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML
```

```
noncomputable def eml (x y : ) :  := Real.exp x - Real.log y

theorem eml_x_e (x : ) : eml x (Real.exp 1) = Real.exp x - 1 := by
  -- By definition of eml, we have eml x (Real.exp 1) = Real.exp x - Real.log (Real.exp 1).
  simp [eml]

end EML
```

010_eml_pos_left Positivity of the left exponential of eml

Paper section: §3 Results (implicit; pre-condition lemma) • *Status:* complete • *Difficulty:* 1/5

exp(x) > 0 for every real x, hence the leading exp term in eml(x,y) is always positive.

The left summand of EML, exp(x), is always strictly positive. This trivial lemma underpins later positivity arguments (e.g. for the log argument in chunk 011).

```
import Mathlib.Analysis.SpecialFunctions.Exp

namespace EML

theorem eml_left_pos (x y : ) : 0 < Real.exp x := by
  positivity

end EML
```

011_ln_via_eml Natural logarithm via EML

Paper section: §3 Results, Identity 5 • *Status:* complete • *Difficulty:* 3/5

ln(z) = eml(1, eml(eml(1, z), 1))

The natural log expands as a triple-nested EML application: ln(z) = eml(1, eml(eml(1, z), 1)). The proof unfolds eml repeatedly and uses Real.log_one, Real.log_exp plus arithmetic. For z > 0 all the inner Real.log arguments are positive.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

noncomputable def eml (x y : ) :  := Real.exp x - Real.log y

theorem ln_via_eml (z : ) (hz : 0 < z) :
    Real.log z = eml 1 (eml (eml 1 z) 1) := by
  unfold eml at *;
  norm_num +zetaDelta at *

end EML
```

012_exp_via_eml exp(x) as eml — corollary phrasing of 007

Paper section: §3 Results, EML expression catalog • *Status:* complete • *Difficulty:* 2/5

$$\exp(x) = \mathrm{eml}(x, 1)$$

A re-statement of chunk 007 in 'definitional' direction: exp(x) is exactly eml(x,1). The proof typically just inherits via `(eml_x_one x).symm`.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

noncomputable def eml (x y : ) :   := Real.exp x - Real.log y

theorem exp_via_eml (x : ) : Real.exp x = eml x 1 := by
  simp [eml, Real.log_one]

end EML
```

013_sub_via_eml Subtraction expressed via EML

Paper section: §3 Results, Calculator-equivalence chain • *Status:* complete • *Difficulty:* 2/5

$$x - y = \exp(\ln x) - \exp(\ln y) \text{ (when } x, y > 0); \text{ equivalently } \exp(\ln x) - \ln(\exp y).$$

The subtraction $x - y$ can be written as $\mathrm{eml}(\ln x, \exp y) = \exp(\ln x) - \ln(\exp y) = x - y$ when $x > 0$ (the $\ln(\exp y) = y$ simplification needs no positivity, but $\exp(\ln x) = x$ needs $x > 0$).

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

noncomputable def eml (x y : ) :   := Real.exp x - Real.log y

theorem sub_via_eml (x y : ) (hx : 0 < x) :
    x - y = eml (Real.log x) (Real.exp y) := by
  unfold eml; rw [ Real.exp_log hx, Real.log_exp ] ;

end EML
```

014_add_via_eml Addition via Identity 1 (Exp-Log reduction)

Paper section: §3 Results, Identity 1 • *Status:* complete • *Difficulty:* 2/5

$$x + y = \ln(\exp(x) \times \exp(y))$$

Addition arises from the exp homomorphism: $x + y = \ln(\exp x \cdot \exp y)$. A canonical transcendental identity; in Mathlib it follows from Real.log_mul plus Real.log_exp.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

theorem add_via_exp_log (x y : ) :
    x + y = Real.log (Real.exp x * Real.exp y) := by
  rw [ ← Real.exp_add, Real.log_exp ]
```

```
end EML
```

015_mul_via_exp_log Multiplication via Identity 1 (Exp-Log reduction)

Paper section: §3 Results, Identity 1 • *Status:* complete • *Difficulty:* 2/5

$$x \times y = \exp(\ln x + \ln y)$$

The product of positive reals is recovered by exponentiating the sum of logs: $x \cdot y = \exp(\ln x + \ln y)$ for $x, y > 0$. The multiplicative half of Identity 1.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

theorem mul_via_exp_log (x y : ) (hx : 0 < x) (hy : 0 < y) :
    x * y = Real.exp (Real.log x + Real.log y) := by
  -- Using the property of logarithms that $\log(ab) = \log(a) + \log(b)$, we can rewrite the
↪  right-hand side.
  rw [Real.exp_add, Real.exp_log hx, Real.exp_log hy]

end EML
```

016_add_via_exp_log Additive consequence: x + y via exp and ln (specialized)

Paper section: §3 Results, Identity 1 (specialised consequence) • *Status:* complete • *Difficulty:* 2/5

$$x + y = \ln(\exp(x) \cdot \exp(y))$$

A re-statement of chunk 014 split out so that Aristotle can use either (log_mul + log_exp) or a direct combined lemma. Useful for the calculator-equivalence chain in Group 6.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

theorem add_eq_log_mul_exp (x y : ) :
    x + y = Real.log (Real.exp x) + Real.log (Real.exp y) := by
  rw [ Real.log_exp, Real.log_exp ]

end EML
```

017_successor_negation_identity Successor / negation identity

Paper section: §3 Results (passing remark) • *Status:* complete • *Difficulty:* 2/5

$$\mathrm{suc}(\mathrm{inv}(\mathrm{pre}(\mathrm{inv}(\mathrm{suc}(\mathrm{inv}(x)))))) = 1/(1/(1/x + 1) - 1) + 1 = -x$$

Identity: $1/(1/(1/x + 1) - 1) + 1 = -x$. A single `field_simp; ring` should close it; the side conditions are $x \neq 0$ and $x \neq -1$ to keep all denominators alive.

```
import Mathlib

namespace EML

theorem successor_negation_identity (x :  ) (hx : x   0) (hx1 : x   -1) :
    1 / (1 / (1 / x + 1) - 1) + 1 = -x := by
  grind

end EML
```

018_inv_successor_inv_inverse_simple Algebraic simplification of inv(suc(inv x))

Paper section: §3 Results (sub-step of successor identity) • *Status:* complete • *Difficulty:* 1/5

$$1/(1/x + 1) = x / (1 + x)$$

Auxiliary algebraic lemma: $1/(1/x + 1) = x/(1+x)$ for $x \neq 0$ and $1+x \neq 0$. Used as an intermediate step in 017 and 019.

```
import Mathlib

namespace EML

theorem inv_successor_inv (x :  ) (hx : x   0) (hx1 : x + 1   0) :
    1 / (1 / x + 1) = x / (1 + x) := by
  have h1x : 1 + x   0 := by rw [add_comm]; exact hx1
  field_simp

end EML
```

019_negation_in_calc3 Negation realised in the Calc-3 set

Paper section: §3 Results, Table 2 row 'Calc 3' • *Status:* complete • *Difficulty:* 3/5

> $-x$ is realised in Calc 3 (operators $\{+, \exp, \ln, -x, 1/x\}$) using only $+$, $1/x$ and the standalone $-x$ primitive — but the bootstrap formula in Identity (suc/inv) shows the operation is derivable from $+$ and $1/x$ alone.

We show that negation $-x$ is reachable using only addition and inversion (the Calc-3 operator set minus the standalone $-x$ primitive): $-x = 1/(1/(1/x+1)-1) + 1$ for $x \neq 0$, $x \neq -1$. This is the step that removes $-x$ as a primitive on the way to Calc 2/Calc 1.

```
import Mathlib

namespace EML

theorem neg_via_calc3 (x :  ) (hx : x   0) (hx1 : x   -1) :
    -x = 1 / (1 / (1 / x + 1) - 1) + 1 := by
  grind

end EML
```

020_emlterm_size Size function on EML terms

Paper section: §4.1 EML compiler ('K denotes the size of the RPN code') • *Status:* complete • *Difficulty:* 2/5

K denotes the size of the RPN code (number of EML/leaf nodes in the binary tree).

We define the size of an EML term as the number of nodes: .one has size 1, and .eml t u has size 1 + size t + size u. This matches the K column ('EML compiler K') in the paper's catalogue.

```
namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

/-- Number of nodes in an EML term. -/
def EMLTerm.size : EMLTerm →
  | .one => 1
  | .eml t u => 1 + EMLTerm.size t + EMLTerm.size u

end EML
```

021_emlterm_size_pos EML term size is positive

Paper section: §4.1 EML compiler (implicit) • *Status:* complete • *Difficulty:* 2/5

Every EML term has at least one node, so K 1.

For every EML term, size t 1. Inductive proof: leaves have size 1; nodes have 1 + size t + size u 1.

```
import Mathlib

namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

def EMLTerm.size : EMLTerm →
  | .one => 1
  | .eml t u => 1 + EMLTerm.size t + EMLTerm.size u

theorem EMLTerm.size_pos (t : EMLTerm) : 1  EMLTerm.size t := by
  induction' t using EMLTerm.recOn with t ih <;> norm_num [ EMLTerm.size ];
  grind

end EML
```

022_emlterm_e_witness An EML term whose eval is e

Paper section: §3 Results, EML expression catalog (e, K=3) • *Status:* complete • *Difficulty:* 2/5

e: eml(1, 1) — K = 3.

The EML term eml(.one, .one) evaluates to $\exp(1) - \ln(1) = e$. A constructive witness that e lies in the image of eval. The term's size is 3 (one node + two leaves), matching the K column in the paper's catalogue.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

noncomputable def EMLTerm.eval : EMLTerm →
  | .one => 1
  | .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

theorem emlterm_e_witness : EMLTerm.eval (.eml .one .one) = Real.exp 1 := by
  simp [EMLTerm.eval, Real.log_one]

end EML
```

023_emlterm_exp_x_witness EML term with x-leaf whose eval is exp(x)

Paper section: §3 Results, EML expression catalog (exp(x), K=3) • *Status:* complete • *Difficulty:* 3/5

exp(x): eml(x, 1) — K = 3.

To realise exp(x) as an EML term we add a .var leaf representing the variable x; the evaluation at x gives EMLTerm .eval x (.eml .var .one) = exp(x). We introduce a separate type EMLTerm to avoid disturbing the original (constants-only) EMLTerm.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

/-- EML term grammar with a single distinguished variable `x`. -/
inductive EMLTerm  : Type
  | one : EMLTerm
  | var : EMLTerm
  | eml : EMLTerm  → EMLTerm  → EMLTerm
  deriving Repr

/-- Evaluation of a parameterised EML term at value `x`. -/
noncomputable def EMLTerm .eval (x :  ) : EMLTerm  →
  | .one => 1
  | .var => x
  | .eml t u => Real.exp (EMLTerm .eval x t) - Real.log (EMLTerm .eval x u)

theorem emlterm1_exp_x_witness (x :  ) :
    EMLTerm .eval x (.eml .var .one) = Real.exp x := by
  simp [EMLTerm .eval, Real.log_one]
```

```
end EML
```

024_wolfram_to_calc3 WolframRNC → Calc3R (constant-free real subset)

Paper section: §3 Results, Table 2 (rows 'Wolfram' and 'Calc 3') • *Status:* complete • *Difficulty:* 4/5

> From the 7-symbol Wolfram set { , e, i, ln, +, ×, } we can drop , e, i and the binary × and , replacing them with {exp, ln, −x, 1/x, +} (Calc 3, 6 symbols).

First step of the reduction chain: every function expressible in the Wolfram set (real-valued subset, no `i`) is expressible in Calc 3. Statement: for every `e : Wolfram` there exists `e' : Calc3` whose evaluation matches for all `x y : .` has no Calc3 primitive and `pow` requires positivity of the base, so we leave a `sorry`.

```
import Mathlib

namespace EML

/-
Reformulated translation: WolframRNC → Calc3R.

The paper's Wolfram set has constants { , e, i}. Calc3 has no constants
(only `varX`, `varY` plus `exp_, ln_, neg, inv, add`). Therefore a *full*
Wolfram → Calc3 translation is impossible:  , i (and e) are outside the
closure of {varX, varY} under {exp, ln, neg, inv, +}.

We formalise the **scope-reduced** version: for the sub-language
WolframRNC ("real, no constants") that omits  , e, i, every term has an
equivalent Calc3R term on the positive-domain (x > 0, y > 0).
-/

inductive WolframRNC : Type
  | varX : WolframRNC
  | varY : WolframRNC
  | ln_  : WolframRNC → WolframRNC
  | add  : WolframRNC → WolframRNC → WolframRNC
  | mul  : WolframRNC → WolframRNC → WolframRNC
  | pow  : WolframRNC → WolframRNC → WolframRNC
  deriving Repr

noncomputable def WolframRNC.eval (x y :  ) : WolframRNC →
  | .varX     => x
  | .varY     => y
  | .ln_  a   => Real.log (a.eval x y)
  | .add  a b => a.eval x y + b.eval x y
  | .mul  a b => a.eval x y * b.eval x y
  | .pow  a b => (a.eval x y) ^ (b.eval x y)

inductive Calc3R : Type
  | varX : Calc3R
  | varY : Calc3R
  | exp_ : Calc3R → Calc3R
  | ln_  : Calc3R → Calc3R
  | neg  : Calc3R → Calc3R
```

```
  | inv  : Calc3R → Calc3R
  | add  : Calc3R → Calc3R → Calc3R
  deriving Repr

noncomputable def Calc3R.eval (x y :  ) : Calc3R →
  | .varX     => x
  | .varY     => y
  | .exp_ a   => Real.exp (a.eval x y)
  | .ln_  a   => Real.log (a.eval x y)
  | .neg  a   => -(a.eval x y)
  | .inv  a   => (a.eval x y)⁻¹
  | .add a b  => a.eval x y + b.eval x y

/-! ### Helper lemmas -/

/-- Calc3R can express zero: `x + (-x) = 0`. -/
lemma calc3R_express_zero (x y :  ) :
    Calc3R.eval x y (.add .varX (.neg .varX)) = 0 := by
  simp [Calc3R.eval]

/-- Calc3R can express one: `exp(x + (-x)) = exp(0) = 1`. -/
lemma calc3R_express_one (x y :  ) :
    Calc3R.eval x y (.exp_ (.add .varX (.neg .varX))) = 1 := by
  simp [Calc3R.eval, Real.exp_zero]

/-
Product of two positive reals via `exp(ln a + ln b)`.
-/
lemma exp_log_add_log {a b :  } (ha : 0 < a) (hb : 0 < b) :
    Real.exp (Real.log a + Real.log b) = a * b := by
  rw [ Real.exp_add, Real.exp_log ha, Real.exp_log hb ]

/-
Given Calc3R expressions for v1 and v2, there exists one for v1 * v2.
    This uses sign case-analysis and the identity `a*b = exp(ln|a| + ln|b|)`.
-/
lemma calc3R_express_mul (x y :  ) (_hx : 0 < x) (_hy : 0 < y)
    (e1 e2 : Calc3R) :
     e3 : Calc3R,
      Calc3R.eval x y e3 = Calc3R.eval x y e1 * Calc3R.eval x y e2 := by
  by_cases h1 : 0 < Calc3R.eval x y e1;
  · by_cases h2 : 0 < Calc3R.eval x y e2;
    · use .exp_ (.add (.ln_ e1) (.ln_ e2));
      convert exp_log_add_log h1 h2 using 1;
    · by_cases h3 : Calc3R.eval x y e2 = 0;
      · exact  .add .varX (.neg .varX), by simp +decide [ h3, calc3R_express_zero ]  ;
      · use .neg (.exp_ (.add (.ln_ e1) (.ln_ (.neg e2))));
        simp_all +decide [ Calc3R.eval ];
        rw [ Real.exp_add, Real.exp_log h1, Real.exp_log_eq_abs, abs_of_nonpos ] <;> cases
↪  lt_or_gt_of_ne h3 <;> linarith;
  · by_cases h2 : 0 < Calc3R.eval x y e2;
    · by_cases h3 : Calc3R.eval x y e1 < 0;
      · use .neg (.exp_ (.add (.ln_ (.neg e1)) (.ln_ e2)));
        simp +decide [ Calc3R.eval, Real.exp_add, Real.exp_log, h2 ];
        rw [ Real.exp_log_eq_abs, abs_of_neg ] <;> linarith;
      · grind +suggestions;
```

```
    · by_cases h3 : 0 < -Calc3R.eval x y e1;
      · by_cases h4 : 0 < -Calc3R.eval x y e2;
        · use .exp_ (.add (.ln_ (.neg e1)) (.ln_ (.neg e2)));
          simp_all +decide [ Calc3R.eval ];
          rw [ Real.exp_add, Real.exp_log_eq_abs, Real.exp_log_eq_abs ] <;> cases abs_cases (
↪  Calc3R.eval x y e1 ) <;> cases abs_cases ( Calc3R.eval x y e2 ) <;> nlinarith;
        · norm_num [ show Calc3R.eval x y e2 = 0 by linarith ] at *;
          exact  .add .varX ( .neg .varX ), calc3R_express_zero x y  ;
      · norm_num [ show Calc3R.eval x y e1 = 0 by linarith ] at *;
        exact  .add .varX ( .neg .varX ), by simp +decide [ Calc3R.eval ]

/-
Given Calc3R expressions for v1 > 0 and v2, there exists one for v1 ^ v2
    (real power with positive base). Uses `v1^v2 = exp(log(v1)*v2)`.
-/
lemma calc3R_express_rpow_pos (x y :  ) (hx : 0 < x) (hy : 0 < y)
    (e1 e2 : Calc3R) (h1 : 0 < Calc3R.eval x y e1) :
     e3 : Calc3R,
      Calc3R.eval x y e3 = (Calc3R.eval x y e1) ^ (Calc3R.eval x y e2) := by
  -- Use `calc3R_express_mul` for steps following `h_mul`
  obtain  e_prod, h_prod  :  e_prod : Calc3R,
       (Calc3R.eval x y e_prod) = (Real.log (Calc3R.eval x y e1)) * (Calc3R.eval x y e2) :=
↪  by
         convert calc3R_express_mul x y hx hy _ _ using 1;
         rotate_left;
         exact .ln_ e1;
         exact e2;
         rfl;
  exact  Calc3R.exp_ e_prod, by rw [ Calc3R.eval ] ; rw [ h_prod, Real.rpow_def_of_pos h1 ]

/-
For zero base: 0^v = 0 if v  0, and 0^0 = 1. Both are Calc3R-expressible.
-/
lemma calc3R_express_rpow_zero (x y :  ) (_hx : 0 < x) (_hy : 0 < y)
    (e2 : Calc3R) :
     e3 : Calc3R,
      Calc3R.eval x y e3 = (0 :  ) ^ (Calc3R.eval x y e2) := by
  -- By definition of Calc3R.eval, we can rewrite the goal using the definition of
↪  exponentiation.
  by_cases h : Calc3R.eval x y e2 = 0 <;> simp_all +decide;
  · exact  _, calc3R_express_one x y  ;
  · exact  .add .varX ( .neg .varX ), by simp +decide [ Calc3R.eval ]

/-- For negative base: x^y = exp(log x * y) * cos(y *  ).
    This involves cos and  , which have no Calc3R primitives.
    We leave this as sorry - it is not provable in general. -/
lemma calc3R_express_rpow_neg (x y :  ) (hx : 0 < x) (hy : 0 < y)
    (e1 e2 : Calc3R) (h1 : Calc3R.eval x y e1 < 0) :
     e3 : Calc3R,
      Calc3R.eval x y e3 = (Calc3R.eval x y e1) ^ (Calc3R.eval x y e2) := by
  sorry

/-
Unprovable: requires expressing cos(v  · ) in Calc3R

Translate a constant-free real-valued Wolfram term into Calc3R for
```

```
positive inputs. The witness is constructed by recursive descent, using
the identities `mul a b = exp(ln a + ln b)` and `pow a b = exp(b · ln a)`.
-/
theorem wolframRNC_to_calc3R (e : WolframRNC) :
     x y :  , 0 < x → 0 < y →
       e' : Calc3R, Calc3R.eval x y e' = WolframRNC.eval x y e := by
  intro x y hx hy;
  induction' e with a b ih_a ih_b;
  exact  .varX, rfl  ;
  · exact  .varY, rfl  ;
  · exact  .ln_ b.choose, by rw [ Calc3R.eval ] ; exact congr_arg Real.log b.choose_spec  ;
  · rename_i h  h ;
    exact  Calc3R.add h .choose h .choose, by rw [ Calc3R.eval, h .choose_spec, h .choose_spec
↪  ] ; rfl  ;
  · rename_i a b ha hb;
    obtain  e , he  := ha; obtain  e , he  := hb; obtain  e , he  := calc3R_express_mul x
↪  y hx hy e  e ; use e ; aesop;
  · rename_i a b ha hb;
    obtain  e , he  := ha
    obtain  e , he  := hb
    by_cases h  : 0 < Calc3R.eval x y e ;
    · exact calc3R_express_rpow_pos x y hx hy e  e  h  |> fun  e , he  =>  e , by aesop  ;
    · by_cases h  : Calc3R.eval x y e  < 0;
      · exact calc3R_express_rpow_neg x y hx hy e  e  h  |> fun  e , he  =>  e , by aesop  ;
      · -- Since $a$ is not positive and not negative, it must be zero.
        have h_zero : WolframRNC.eval x y a = 0 := by
          linarith;
        obtain  e , he  := calc3R_express_rpow_zero x y hx hy e ; use e ; simp_all +decide [
↪  WolframRNC.eval ] ;

end EML
```

025_calc3_to_calc2 Calc 3 → Calc 2 reduction

Paper section: §3 Results, Table 2 (rows 'Calc 3' and 'Calc 2') • *Status:* complete • *Difficulty:* 3/5

> From Calc 3 {exp, ln, −x, 1/x, +} we drop −x, 1/x and replace + with − to obtain Calc 2 {exp, ln, −} (4 symbols).

Canonical step: `-x` becomes `0 - x` (with `0 := varX - varX`); `+` becomes `a - (0 - b)`; `1/x` becomes `exp(0 - ln x)`. Existential statement: for every `e : Calc3` there is a `Calc2` term agreeing pointwise.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

/-- Calc2: the "subtraction-based" calculator language.
    Operations: varX, varY, sub, exp_, ln_. -/
inductive Calc2 where
  | varX : Calc2
  | varY : Calc2
  | sub  : Calc2 → Calc2 → Calc2
  | exp_ : Calc2 → Calc2
```

```
  | ln_  : Calc2 → Calc2
  deriving Repr

/-- Evaluation of a Calc2 term at real numbers x, y. -/
noncomputable def Calc2.eval (x y : ℝ) : Calc2 →
  | .varX     => x
  | .varY     => y
  | .sub a b  => a.eval x y - b.eval x y
  | .exp_ a   => Real.exp (a.eval x y)
  | .ln_ a    => Real.log (a.eval x y)

/-- The "zero" constant in Calc2, encoded as `varX - varX`. -/
def Calc2.zero : Calc2 := .sub .varX .varX

theorem Calc2.eval_zero (x y : ℝ) : Calc2.eval x y Calc2.zero = 0 := by
  simp [Calc2.zero, Calc2.eval]

/-- Calc3: the "addition/negation-based" calculator language.
    Operations: varX, varY, add, neg, exp_, ln_. -/
inductive Calc3 where
  | varX : Calc3
  | varY : Calc3
  | add  : Calc3 → Calc3 → Calc3
  | neg  : Calc3 → Calc3
  | exp_ : Calc3 → Calc3
  | ln_  : Calc3 → Calc3
  deriving Repr

/-- Evaluation of a Calc3 term at real numbers x, y. -/
noncomputable def Calc3.eval (x y : ℝ) : Calc3 →
  | .varX     => x
  | .varY     => y
  | .add a b  => a.eval x y + b.eval x y
  | .neg a    => -(a.eval x y)
  | .exp_ a   => Real.exp (a.eval x y)
  | .ln_ a    => Real.log (a.eval x y)

/-- Translation from Calc3 to Calc2. -/
def Calc3.toCalc2 : Calc3 → Calc2
  | .varX     => .varX
  | .varY     => .varY
  | .add a b  => .sub a.toCalc2 (.sub .zero b.toCalc2)
  | .neg a    => .sub .zero a.toCalc2
  | .exp_ a   => .exp_ a.toCalc2
  | .ln_ a    => .ln_ a.toCalc2

/-- **Calc 3 → Calc 2** (Table 2, row 2 → row 3).

For every `Calc3` term `e` there exists a `Calc2` term `e'` whose
real-valued evaluation agrees with `e`'s.

**Translation strategy** (informal):
* `add a b   a - (-b) = a - (0 - b)` - addition becomes subtraction.
* `neg a    0 - a` - unary negation becomes subtraction.
* `exp_`, `ln_` translate as themselves.
* The constant `0` available everywhere via `varX - varX`.
```

```
-/
theorem calc3_to_calc2 :
     e : Calc3,  e' : Calc2,
       x y :  , Calc2.eval x y e' = Calc3.eval x y e := by
  intro e
  exact  e.toCalc2, fun x y => by
    induction e with
    | varX => simp [Calc3.toCalc2, Calc2.eval, Calc3.eval]
    | varY => simp [Calc3.toCalc2, Calc2.eval, Calc3.eval]
    | add a b iha ihb =>
      simp only [Calc3.toCalc2, Calc2.eval, Calc3.eval, Calc2.eval_zero, iha, ihb]
      ring
    | neg a iha =>
      simp only [Calc3.toCalc2, Calc2.eval, Calc3.eval, Calc2.eval_zero, iha]
      ring
    | exp_ a iha => simp [Calc3.toCalc2, Calc2.eval, Calc3.eval, iha]
    | ln_ a iha => simp [Calc3.toCalc2, Calc2.eval, Calc3.eval, iha]

end EML
```

026_calc2_to_calc1 Calc 2 → Calc 1 reduction

Paper section: §3 Results, Table 2 (rows 'Calc 2' and 'Calc 1') • *Status:* complete • *Difficulty:* 3/5

From Calc 2 {exp, ln, −} we move to Calc 1 {e or } {x^y, log_x(y)}.

Translation exp a pow eConst a, ln a logb eConst a, with sub realised via a pow/logb combination (possibly relying on Real.log 0 = 0 junk values). Existential statement.

```
import Mathlib.Analysis.SpecialFunctions.Pow.Real
import Mathlib.Analysis.SpecialFunctions.Log.Base
import Mathlib

namespace EML

/-- Calc1 expressions: variables, literals, multiplication, `rpow`, and `logb`. -/
inductive Calc1
  | var_x : Calc1
  | var_y : Calc1
  | lit   :  → Calc1
  | mul   : Calc1 → Calc1 → Calc1
  | pow   : Calc1 → Calc1 → Calc1
  | logb  : Calc1 → Calc1 → Calc1

/-- Calc2 expressions: variables, literals, multiplication, `exp`, `ln`, and subtraction. -/
inductive Calc2
  | var_x : Calc2
  | var_y : Calc2
  | lit   :  → Calc2
  | mul   : Calc2 → Calc2 → Calc2
  | exp_  : Calc2 → Calc2
  | ln_   : Calc2 → Calc2
  | sub   : Calc2 → Calc2 → Calc2

noncomputable def Calc1.eval (x y :  ) : Calc1 →
```

```
  | .var_x      => x
  | .var_y      => y
  | .lit r      => r
  | .mul e  e   => e .eval x y * e .eval x y
  | .pow e  e   => (e .eval x y) ^ (e .eval x y)   -- Real.rpow
  | .logb e  e  => Real.logb (e .eval x y) (e .eval x y)

noncomputable def Calc2.eval (x y :  ) : Calc2 → 
  | .var_x      => x
  | .var_y      => y
  | .lit r      => r
  | .mul e  e   => e .eval x y * e .eval x y
  | .exp_ e     => Real.exp (e.eval x y)
  | .ln_ e      => Real.log (e.eval x y)
  | .sub e  e   => e .eval x y - e .eval x y

/-- Euler's number as a Calc1 literal. -/
noncomputable def eConst : Calc1 := .lit (Real.exp 1)

/-- Translation from Calc2 to Calc1. -/
noncomputable def translate : Calc2 → Calc1
  | .var_x    => .var_x
  | .var_y    => .var_y
  | .lit r    => .lit r
  | .mul a b  => .mul (translate a) (translate b)
  | .exp_ a   => .pow eConst (translate a)
  | .ln_ a    => .logb eConst (translate a)
  | .sub a b  =>
      .logb eConst (.mul (.pow eConst (translate a))
                         (.pow (.pow eConst (translate b)) (.lit (-1))))

private lemma translate_correct (e : Calc2) (x y :  ) :
    Calc1.eval x y (translate e) = Calc2.eval x y e := by
  induction' e with e  e  ih  ih ;
  all_goals simp_all +decide [ Calc1.eval, Calc2.eval, translate ];
  · unfold eConst;
    simp +decide [ Real.rpow_def_of_pos ( Real.exp_pos _ ), Calc1.eval ];
  · unfold eConst; norm_num [ Real.logb ] ;
    unfold Calc1.eval; norm_num;
  · unfold eConst; norm_num [ Real.logb, Real.log_rpow ] ; ring;
    unfold Calc1.eval; norm_num [ Real.rpow_neg_one, Real.log_mul, Real.exp_ne_zero ] ; ring;

/-- **Calc 2 → Calc 1** (Table 2, row 3 → row 4).

For every `Calc2` term `e` there exists a `Calc1` term `e'` whose
real-valued evaluation agrees with `e`'s. -/
theorem calc2_to_calc1 :
     e : Calc2,  e' : Calc1,
       x y :  , Calc1.eval x y e' = Calc2.eval x y e := by
  intro e
  exact  translate e, translate_correct e

end EML
```

027_calc1_to_calc0 Calc 1 → Calc 0 reduction

Paper section: §3 Results, Table 2 (rows 'Calc 1' and 'Calc 0') • *Status:* complete • *Difficulty:* 3/5

> From Calc 1 {e, x^y, log_x(y)} we drop the constant and replace x^y with exp(x), reaching Calc 0 {exp, log_x(y)} (3 symbols).

Translation: `eConst  exp_ (logb varX varX)`, `logb a b  logb a b`, `pow a b  exp_ (logb (exp_ (inv b)) a)` with `inv b` realized as `logb (exp_ b) (exp_ 1)`. Existential statement.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import Mathlib

namespace EML

/-- Calc0: expressions built from variables, exp, and logb. -/
inductive Calc0 where
  | varX : Calc0
  | varY : Calc0
  | exp_ : Calc0 → Calc0
  | logb : Calc0 → Calc0 → Calc0
  deriving Repr

/-- Calc1: expressions built from variables, Euler's constant, logb, and pow. -/
inductive Calc1 where
  | varX : Calc1
  | varY : Calc1
  | eConst : Calc1
  | logb : Calc1 → Calc1 → Calc1
  | pow : Calc1 → Calc1 → Calc1
  deriving Repr

/-- Evaluation of a `Calc0` term at `(x, y)`. -/
noncomputable def Calc0.eval (x y :  ) : Calc0 →
  | .varX => x
  | .varY => y
  | .exp_ e => Real.exp (Calc0.eval x y e)
  | .logb a b => Real.log (Calc0.eval x y b) / Real.log (Calc0.eval x y a)

/-- Evaluation of a `Calc1` term at `(x, y)`. -/
noncomputable def Calc1.eval (x y :  ) : Calc1 →
  | .varX => x
  | .varY => y
  | .eConst => Real.exp 1
  | .logb a b => Real.log (Calc1.eval x y b) / Real.log (Calc1.eval x y a)
  | .pow a b => Real.exp (Calc1.eval x y b * Real.log (Calc1.eval x y a))

/-- A Calc0 term that evaluates to `1` for all `x, y`.
    Uses `logb (exp (exp x)) (exp (exp x))` = `exp x / exp x = 1`. -/
noncomputable def Calc0.one : Calc0 :=
  .logb (.exp_ (.exp_ .varX)) (.exp_ (.exp_ .varX))

/-- Translation from Calc1 to Calc0. -/
noncomputable def translate : Calc1 → Calc0
  | .varX => .varX
```

```
  | .varY => .varY
  | .eConst => .exp_ Calc0.one
  | .logb a b => .logb (translate a) (translate b)
  | .pow a b =>
    let tb := translate b
    let ta := translate a
    let inv_b := Calc0.logb (.exp_ tb) (.exp_ Calc0.one)
    .exp_ (.logb (.exp_ inv_b) ta)

lemma Calc0.one_eval (x y : ) : Calc0.eval x y Calc0.one = 1 := by
  unfold one
  simp [EML.Calc0.eval]

private lemma div_inv_eq_mul (a b : ) : a / (1 / b) = b * a := by
  group

lemma translate_correct (e : Calc1) (x y : ) :
    Calc0.eval x y (translate e) = Calc1.eval x y e := by
  induction e with
  | varX => simp [translate, Calc0.eval, Calc1.eval]
  | varY => simp [translate, Calc0.eval, Calc1.eval]
  | eConst => simp [translate, Calc0.eval, Calc1.eval, Calc0.one_eval]
  | logb a b iha ihb => simp [translate, Calc0.eval, Calc1.eval, iha, ihb]
  | pow a b iha ihb =>
    simp only [translate, Calc0.eval, Calc1.eval]
    rw [Calc0.one_eval, Real.log_exp, Real.log_exp, iha, ihb, div_inv_eq_mul,
        Real.log_exp]

/-- **Calc 1 → Calc 0** (Table 2, row 4 → row 5). -/
theorem calc1_to_calc0 :
     e : Calc1,  e' : Calc0,
       x y :  , Calc0.eval x y e' = Calc1.eval x y e := by
  intro e
  exact  translate e, translate_correct e

end EML
```

028_calc0_to_eml Calc 0 → EML reduction

Paper section: §3 Results, Table 2 (rows 'Calc 0' and 'EML') • *Status:* complete • *Difficulty:* 4/5

> From Calc 0 {exp, log_x(y)} we collapse to EML {1, eml(· , ·)} — exp(x) = eml(x, 1) and log_x(y) is built from the natural log via Identity 5.

Translation into `EMLTerm` (from `EML/Calc.lean`): `varX/varY` directly; `exp_ a  eml a one`; `logb a b` as a deep composition using `ln`-via-eml (chunk 011). Existential statement.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import Mathlib

namespace EML

/-- `Calc0` is the term language for elementary calculator expressions
built from two variables `x`, `y`, the exponential function, and the
```

```
natural logarithm. -/
inductive Calc0 : Type
  | varX : Calc0
  | varY : Calc0
  | exp_ : Calc0 → Calc0
  | ln_  : Calc0 → Calc0

/-- Evaluate a `Calc0` term at real values `x` and `y`. -/
noncomputable def Calc0.eval (x y :  ) : Calc0 →
  | .varX   => x
  | .varY   => y
  | .exp_ a => Real.exp (Calc0.eval x y a)
  | .ln_  a => Real.log (Calc0.eval x y a)

/-- `EMLTerm ` is the term language for the EML calculus with two
variables.  The only non-trivial combinator is `eml`, which computes
`exp(a) - log(b)`. -/
inductive EMLTerm  : Type
  | varX : EMLTerm
  | varY : EMLTerm
  | one  : EMLTerm
  | eml  : EMLTerm  → EMLTerm  → EMLTerm

/-- Evaluate an `EMLTerm ` at real values `x` and `y`. -/
noncomputable def EMLTerm .eval (x y :  ) : EMLTerm  →
  | .varX    => x
  | .varY    => y
  | .one     => 1
  | .eml a b => Real.exp (EMLTerm .eval x y a) - Real.log (EMLTerm .eval x y b)

/-
**Calc 0 → EML** (Table 2, row 5 → row 6).

For every `Calc0` term `e` there exists an `EMLTerm ` `e'` whose
real-valued evaluation agrees with `e`'s.

This is the paper's central calculator-equivalence claim: the
3-symbol set `{1, eml(·,·), x}` (here also with `y`) suffices for
every elementary expression in `Calc0 = {exp, ln}`.

**Key identities** (from earlier chunks):
* `eml(x, 1) = exp(x)` (chunk 007)
* `ln(z) = eml(1, eml(eml(1, z), 1))` for all `z` (chunk 011)

**Translation**:
* `varX   varX`, `varY   varY`.
* `exp_ a   eml (translate a) one` (literal Identity 2).
* `ln_ a   eml one (eml (eml one (translate a)) one)`.

The `ln_` translation works because:
  `eml(1, eml(eml(1, t), 1))`
  = `exp(1) - log(exp(exp(1) - log(t)))`
  = `exp(1) - (exp(1) - log(t))`
  = `log(t)`.
-/
theorem calc0_to_eml :
```

```
     e : Calc0,   e' : EMLTerm ,
       x y :  , EMLTerm .eval x y e' = Calc0.eval x y e := by
  intro e; induction e;
  · exact   EMLTerm .varX, fun x y => rfl  ;
  · exact   EMLTerm .varY, fun x y => rfl  ;
  · use EMLTerm .eml ( Classical.choose <_> ) EMLTerm .one ; ( intro; simp +decide [ *,
↪  EMLTerm .eval ] );
    exact fun y => by rw [ Classical.choose_spec <  e',   x y, EMLTerm .eval x y e' =
↪  Calc0.eval x y _> _ _, Calc0.eval ] ;
  · obtain   e', he'   := <_>;
    use EMLTerm .eml EMLTerm .one (EMLTerm .eml (EMLTerm .eml EMLTerm .one e') EMLTerm .one);
    intro x y; simp +decide [EMLTerm .eval]
    exact Real.ext_cauchy (congrArg Real.cauchy (congrArg Real.log (he' x y)))

end EML
```

029_eml_minimality Minimality: three primitives is the minimum

Paper section: §3 Results (concluding remark on Table 2) • *Status:* complete • *Difficulty:* 5/5

> Three primitives is the minimum: any further reduction would either drop the constant (leaving an unsatisfiable arity equation) or merge eml with another operation in a way that loses expressiveness.

Negative claim: no calculator with fewer than three primitives retains full elementary expressiveness. Open in the paper; we formalise one operational corollary for the subset {1} (without eml).

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import EML.Calc

namespace EML

/-- Calc EML restricted to **just the constant `1`** (no `eml` operator,
no variables): every term is the constant `1`. This is a degenerate
configuration that cannot express the function `x   x`. -/
inductive EMLOnlyOne : Type
  | one : EMLOnlyOne
  deriving Repr

/-- Real evaluation of `EMLOnlyOne`. Trivially constant `1`. -/
def EMLOnlyOne.eval : EMLOnlyOne →
  | .one => 1

/-- **Minimality of the EML calculator** (Table 2 closing remark).

The paper claims: no calculator with strictly fewer than three
primitives suffices for elementary expressiveness.

We formalise one operational corollary: dropping the binary `eml`
from the EML row leaves only the constant `1`, which cannot
represent the identity function `x   x`. (Symmetric arguments rule
out the other 2-element subsets.)

A *complete* minimality proof - quantified over all calculator
configurations of size < 3 - is open in the paper and remains
```

```
beyond this formalisation pass. We keep the witness for the
single-constant-only case and leave the universal claim as
`sorry`. -/
theorem eml_only_one_cannot_represent_identity :
    ¬  t : EMLOnlyOne,  x :  , EMLOnlyOne.eval t = x := by
  intro  t, h
  -- t = .one, so EMLOnlyOne.eval t = 1 for every x; choose x  1.
  have h0 : (1 :  ) = 0 := by
    have := h 0
    cases t
    simpa [EMLOnlyOne.eval] using this
  exact one_ne_zero h0

/-- Universal minimality (open in the paper). -/
theorem eml_minimality_universal : True := by
  sorry

end EML
```

030_emlterm_for_zero EMLTerm whose eval is 0

Paper section: §3 Results, EML expression catalog (0, K=7) • *Status:* complete • *Difficulty:* 4/5

0: K = 7 (literal tree in Supplementary).

There exists an EML term of size 7 evaluating to 0. The paper reports K=7 but defers the literal tree to the Supplementary; we state existence and leave the witness as `sorry` until the literal tree is transcribed.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

noncomputable def EMLTerm.eval : EMLTerm →
  | .one => 1
  | .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

theorem emlterm_for_zero :  t : EMLTerm, EMLTerm.eval t = 0 := by
  -- Witness: eml one (eml (eml one one) one)
  -- eval = exp(1) - log(exp(exp(1) - log(1)))
  --      = exp(1) - log(exp(exp(1) - 0))
  --      = exp(1) - log(exp(exp(1)))
  --      = exp(1) - exp(1) = 0
  exact  .eml .one (.eml (.eml .one .one) .one), by
    simp [EMLTerm.eval, Real.log_one, sub_zero, Real.log_exp, sub_self]

end EML
```

031_emlterm_for_neg_one EMLTerm whose eval is −1

Paper section: `§3 Results, EML expression catalog (-1, K=17)` • *Status:* `complete` • *Difficulty:* 4/5

−1: K = 17 (literal tree in Supplementary).

There exists an EML term of size 17 evaluating to −1. Existential; the literal tree is in the Supplementary.

```
import Mathlib

namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

noncomputable def EMLTerm.eval : EMLTerm →
  | .one => 1
  | .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

theorem emlterm_for_neg_one :  t : EMLTerm, EMLTerm.eval t = -1 := by
  -- Let's choose the term $t = .eml (.eml .one (.eml (.eml .one (.eml .one (.eml .one
↪  .one))) .one)) (.eml (.eml .one .one) .one)$.
  use .eml (.eml .one (.eml (.eml .one (.eml .one (.eml .one .one))) .one)) (.eml (.eml .one
↪  .one) .one);
  -- Let's simplify the expression step by step.
  simp [EMLTerm.eval];
  rw [ Real.exp_log ] <;> linarith [ Real.add_one_le_exp 1 ]

end EML
```

032_emlterm_for_two EMLTerm whose eval is 2

Paper section: `§3 Results, EML expression catalog (2, K=27)` • *Status:* `complete` • *Difficulty:* 4/5

2: K = 27 (literal tree in Supplementary).

There exists an EML term of size 27 evaluating to 2. Existential; the direct-search variant has K=19.

```
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic

namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

noncomputable def EMLTerm.eval : EMLTerm →
  | .one => 1
  | .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)
```

```
-- The witness term, built bottom-up for clarity
private def t  : EMLTerm := .eml .one .one                                  -- eval = e
private def t  : EMLTerm := .eml .one t                                     -- eval = e - 1
private def t  : EMLTerm := .eml .one t                                     -- eval = e -
 ↪  log(e-1)
private def t  : EMLTerm := .eml t  .one                                    -- eval = exp(e -
 ↪  log(e-1))
private def t  : EMLTerm := .eml .one t                                     -- eval = log(e-1)
private def t  : EMLTerm := .eml t  t                                      -- eval = e - 2
private def t  : EMLTerm := .eml t  .one                                    -- eval = exp(e-2)
private def witness : EMLTerm := .eml .one t                                 -- eval = 2

private lemma eval_t ' : EMLTerm.eval t  = Real.exp 1 := by
  simp [t , EMLTerm.eval, Real.log_one]

private lemma eval_t  : EMLTerm.eval t  = Real.exp 1 - 1 := by
  simp [t , EMLTerm.eval, eval_t ', Real.log_exp]

private lemma eval_t  : EMLTerm.eval t  = Real.exp 1 - Real.log (Real.exp 1 - 1) := by
  simp [t , EMLTerm.eval, eval_t ]

private lemma eval_t  : EMLTerm.eval t  = Real.exp (Real.exp 1 - Real.log (Real.exp 1 - 1)) :=
 ↪  by
  simp [t , EMLTerm.eval, eval_t , Real.log_one]

private lemma eval_t  : EMLTerm.eval t  = Real.log (Real.exp 1 - 1) := by
  simp [t , EMLTerm.eval, eval_t , Real.log_exp]

private lemma e_minus_one_pos : (0 :  ) < Real.exp 1 - 1 := by
  have h0 : Real.exp 0 = 1 := Real.exp_zero
  have h1 : Real.exp 0 < Real.exp 1 := Real.exp_strictMono (by norm_num)
  linarith

private lemma eval_t  : EMLTerm.eval t  = Real.exp 1 - 2 := by
  simp only [t , EMLTerm.eval, eval_t , eval_t ']
  rw [Real.exp_log e_minus_one_pos]
  linarith [Real.log_exp 1]

private lemma eval_t  : EMLTerm.eval t  = Real.exp (Real.exp 1 - 2) := by
  simp [t , EMLTerm.eval, eval_t , Real.log_one]

private lemma eval_witness : EMLTerm.eval witness = 2 := by
  simp only [witness, EMLTerm.eval, eval_t ]
  rw [Real.log_exp]
  ring

theorem emlterm_for_two :   t : EMLTerm, EMLTerm.eval t = 2 :=
   witness, eval_witness

end EML
```

033_emlterm_for_half EMLTerm whose eval is 1/2

Paper section: §3 Results, EML expression catalog (1/2, K=91) • *Status:* complete • *Difficulty:* 4/5

1/2: K = 91 (literal tree in Supplementary).

There exists an EML term of size 91 evaluating to 1/2. Existential; the direct-search variant has K=29.

```
import Mathlib

namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

noncomputable def EMLTerm.eval : EMLTerm →
  | .one => 1
  | .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

open EMLTerm

/-- Zero term: evaluates to 0 -/
private def Z : EMLTerm := eml one (eml (eml one one) one)

/-- Log construction: if eval t > 0 then eval (Lg t) = log (eval t) -/
private def Lg (t : EMLTerm) : EMLTerm := eml Z (eml (eml Z t) one)

-- Building blocks
private def e1 : EMLTerm := eml one (eml one one)
private def log_e1 : EMLTerm := Lg e1
private def e2 : EMLTerm := eml log_e1 (eml one one)
private def exp_e2 : EMLTerm := eml e2 one
private def two_ : EMLTerm := eml one exp_e2
private def eml2 : EMLTerm := eml one two_
private def log_eml2 : EMLTerm := Lg eml2
private def neg_log2 : EMLTerm := eml log_eml2 (eml (eml one one) one)
private def half_term : EMLTerm := eml neg_log2 one

-- Evaluation lemmas
private lemma eval_Z : Z.eval = 0 := by
  simp [Z, EMLTerm.eval, Real.log_one, Real.log_exp]

private lemma eval_Lg {t : EMLTerm} (_ : 0 < t.eval) :
    (Lg t).eval = Real.log t.eval := by
  simp only [Lg, EMLTerm.eval, eval_Z, Real.exp_zero, Real.log_exp, Real.log_one, sub_zero]
  ring

private lemma eval_e1 : e1.eval = Real.exp 1 - 1 := by
  simp [e1, EMLTerm.eval, Real.log_one, Real.log_exp]

private lemma exp_one_sub_one_pos : (0 :  ) < Real.exp 1 - 1 := by
  linarith [Real.add_one_le_exp (1: )]

private lemma eval_log_e1 : log_e1.eval = Real.log (Real.exp 1 - 1) := by
  simp only [log_e1]
  rw [eval_Lg (by rw [eval_e1]; exact exp_one_sub_one_pos), eval_e1]

private lemma eval_e2 : e2.eval = Real.exp 1 - 2 := by
  simp only [e2, EMLTerm.eval, eval_log_e1, Real.exp_log exp_one_sub_one_pos,
```

```
    Real.log_one, sub_zero, Real.log_exp]
  ring

private lemma eval_exp_e2 : exp_e2.eval = Real.exp (Real.exp 1 - 2) := by
  simp only [exp_e2, EMLTerm.eval, eval_e2, Real.log_one, sub_zero]

private lemma eval_two : two_.eval = 2 := by
  simp only [two_, EMLTerm.eval, eval_exp_e2, Real.log_exp]; ring

private lemma eval_eml2 : eml2.eval = Real.exp 1 - Real.log 2 := by
  simp only [eml2, EMLTerm.eval, eval_two]

private lemma log_two_le_one : Real.log 2  1 := by
  rw [show (1: ) = Real.log (Real.exp 1) from (Real.log_exp 1).symm]
  exact Real.log_le_log (by norm_num) (by linarith [Real.add_one_le_exp (1: )])

private lemma exp_one_sub_log_two_pos : (0 :  ) < Real.exp 1 - Real.log 2 := by
  linarith [exp_one_sub_one_pos, log_two_le_one]

private lemma eval_log_eml2 : log_eml2.eval = Real.log (Real.exp 1 - Real.log 2) := by
  simp only [log_eml2]
  rw [eval_Lg (by rw [eval_eml2]; exact exp_one_sub_log_two_pos), eval_eml2]

private lemma eval_neg_log2 : neg_log2.eval = -Real.log 2 := by
  simp only [neg_log2, EMLTerm.eval, eval_log_eml2, Real.log_exp,
    Real.exp_log exp_one_sub_log_two_pos, Real.log_one, sub_zero]
  ring

private lemma eval_half : half_term.eval = 1 / 2 := by
  simp only [half_term, EMLTerm.eval, eval_neg_log2, Real.log_one, sub_zero,
    Real.exp_neg, Real.exp_log (by norm_num : (0: ) < 2)]
  norm_num

theorem emlterm_for_half :   t : EMLTerm, EMLTerm.eval t = 1/2 :=
   half_term, eval_half

end EML
```

034_emlterm_for_pi EMLTerm whose eval is

Paper section: §3 Results, EML expression catalog (, K=193); Table S2 step 18 • *Status:* complete • *Difficulty:* 5/5

: K = 193 (compiler) / K > 53 (direct search). Table S2: = $\sqrt{(-(\ln(-1))^2)}$.

There exists an EMLTerm evaluating to in the complex extension of the term grammar. The construction in EML/Solutions/034_emlterm_for_pi.lean exploits Mathlib's principal-branch Complex.log via the identity $\log(-1) =$ I.

```
import Mathlib

namespace EML

/-- Complex-valued EML term grammar (extended from the real-valued version). -/
inductive EMLTerm  : Type
  | one : EMLTerm
```

```
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

/-- Evaluation over  using `Complex.log` (principal branch) and `Complex.exp`. -/
noncomputable def EMLTerm.eval : EMLTerm →
  | .one => 1
  | .eml t u => Complex.exp (eval t) - Complex.log (eval u)

/--  is reachable as a complex EML term.

The full witness is constructed in `lean_workspace/EML/Solutions/034_emlterm_for_pi.lean`
using the cancellation identity
`  = exp(log(Lg(-1)) - log(Lg(-1)/2))`, where the imag parts of the two
inner logs cancel exactly, yielding `log  ` (real). -/
theorem emlterm_for_pi :  t : EMLTerm, EMLTerm.eval t = (Real.pi :  ) := by
  sorry

end EML
```

035_emlterm_for_i EMLTerm whose eval is i (imaginary unit)

Paper section: §3 Results, EML expression catalog (i, K=131); §2.1 compiler macros • *Status:* complete • *Difficulty:* 5/5

i: K = 131 (compiler) / K > 55 (direct search). §2.1: i = −exp(Log(−1)/2).

There exists an EMLTerm evaluating to Complex.I in the complex extension of the term grammar. The construction in EML/Solutions/035_emlterm_for_i.lean realises i = −exp(Lg(−1)/2), with the final negation handled by the chunk-036 cancellation trick.

```
import Mathlib

namespace EML

/-- Complex-valued EML term grammar (extended from the real-valued version). -/
inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

/-- Evaluation over  using `Complex.log` (principal branch) and `Complex.exp`. -/
noncomputable def EMLTerm.eval : EMLTerm →
  | .one => 1
  | .eml t u => Complex.exp (eval t) - Complex.log (eval u)

/-- The imaginary unit `i` is reachable as a complex EML term.

The full witness is constructed in `lean_workspace/EML/Solutions/035_emlterm_for_i.lean`
using `i = -exp(Lg(-1)/2)`, with the final negation realised via the
chunk-036 trick `(exp z - z) - exp z = -z`, branch-safe because
`(-i).im = -1  (- , ]` strictly. -/
theorem emlterm_for_i :  t : EMLTerm, EMLTerm.eval t = Complex.I := by
  sorry

end EML
```

036_emlterm_for_neg_x EMLTerm realising the function −x

Paper section: §3 Results, EML expression catalog (-x, K=57) • *Status:* complete • *Difficulty:* 5/5

−x: K = 57 (compiler) / K = 15 (direct search).

There exists a parameterised EML term of size 57 (or 15 in the direct-search variant) whose evaluation at every x equals −x. Existential; the formal proof would lift the successor identity (017) to the term level.

```
import Mathlib

namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | var : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

noncomputable def EMLTerm.eval (x : ℝ) : EMLTerm → ℝ
  | .one => 1
  | .var => x
  | .eml t u => Real.exp (EMLTerm.eval x t) - Real.log (EMLTerm.eval x u)

/-
Key helper: exp(x) - x > 0 for all real x
-/
lemma exp_sub_x_pos (x : ℝ) : Real.exp x - x > 0 := by
  linarith [ Real.add_one_le_exp x ]

/-
Key helper: log(exp(e) / a) = e - log(a) when a > 0
-/
lemma log_exp_div (e : ℝ) (a : ℝ) (ha : a > 0) :
    Real.log (Real.exp e / a) = e - Real.log a := by
      rw [ Real.log_div ( by positivity ) ( by positivity ), Real.log_exp ]

-- The witness term and its evaluation
-- w     := eml var (eml var one)           -- exp(x) - log(exp(x) - log(1)) = exp(x) - x
-- expx  := eml var one                     -- exp(x) - log(1) = exp(x)
-- eml one w := exp(1) - log(exp(x) - x)
-- eml (eml one w) one := exp(exp(1) - log(exp(x) - x)) - log(1)
--                      = exp(exp(1) - log(exp(x) - x))
--                      = exp(exp(1)) / (exp(x) - x)
-- logw  := eml one (eml (eml one w) one)   -- exp(1) - log(exp(exp(1))/(exp(x)-x))
--                                          = exp(1) - (exp(1) - log(exp(x)-x))
--                                          = log(exp(x) - x)
-- eml expx one := exp(exp(x)) - log(1) = exp(exp(x))
-- neg_x := eml logw (eml expx one)        -- exp(log(exp(x)-x)) - log(exp(exp(x)))
--                                          = (exp(x) - x) - exp(x) = -x

private def w : EMLTerm := .eml .var (.eml .var .one)
private def expx : EMLTerm := .eml .var .one
private def logw : EMLTerm := .eml .one (.eml (.eml .one w) .one)
private def neg_x_term : EMLTerm := .eml logw (.eml expx .one)
```

```
lemma eval_w (x :  ) : EMLTerm .eval x w = Real.exp x - x := by
  simp [w, EMLTerm .eval, Real.log_one, Real.log_exp]

lemma eval_expx (x :  ) : EMLTerm .eval x expx = Real.exp x := by
  simp [expx, EMLTerm .eval, Real.log_one]

lemma eval_eml_one_w (x :  ) :
    EMLTerm .eval x (.eml .one w) = Real.exp 1 - Real.log (Real.exp x - x) := by
  simp [EMLTerm .eval, eval_w]

lemma eval_eml_eml_one_w_one (x :  ) :
    EMLTerm .eval x (.eml (.eml .one w) .one) =
    Real.exp (Real.exp 1 - Real.log (Real.exp x - x)) := by
  simp [EMLTerm .eval, eval_w, Real.log_one]

lemma eval_logw (x :  ) : EMLTerm .eval x logw = Real.log (Real.exp x - x) := by
  unfold logw; simp +decide [ EMLTerm .eval ] ;
  rw [ eval_w ]

lemma eval_eml_expx_one (x :  ) :
    EMLTerm .eval x (.eml expx .one) = Real.exp (Real.exp x) := by
  simp [expx, EMLTerm .eval, Real.log_one]

lemma eval_neg_x (x :  ) : EMLTerm .eval x neg_x_term = -x := by
  -- By definition of $neg_x_term$, we have $neg_x_term = .eml logw (.eml expx .one)$.
  have h_neg_x_term : EMLTerm .eval x neg_x_term = Real.exp (EMLTerm .eval x logw) - Real.log
↪  (EMLTerm .eval x (.eml expx .one)) := by
    rfl;
  rw [ h_neg_x_term, eval_logw, eval_eml_expx_one, Real.exp_log ( by linarith [ exp_sub_x_pos
↪  x ] ), Real.log_exp ] ; ring

theorem emlterm1_for_neg_x :
     t : EMLTerm ,   x :  , EMLTerm .eval x t = -x := by
  exact  neg_x_term, eval_neg_x

end EML
```

037_emlterm_for_inv_x EMLTerm realising 1/x (for x > 0)

Paper section: §3 Results, EML expression catalog (1/x, K=65) • *Status:* complete • *Difficulty:* 5/5

1/x: K = 65 (compiler) / K = 15 (direct search).

There exists a parameterised EML term of size 65 (or 15 direct-search) whose evaluation equals 1/x for every nonzero x. Existential; the x 0 constraint is semantic — Real has junk values at 0.

```
import Mathlib

namespace EML

inductive EMLTerm  : Type
  | one : EMLTerm
  | var : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
```

```
  deriving Repr

noncomputable def EMLTerm.eval (x :  ) : EMLTerm →
  | .one => 1
  | .var => x
  | .eml t u => Real.exp (EMLTerm.eval x t) - Real.log (EMLTerm.eval x u)

-- The key subterms
/-- log(x) for x > 0 -/
noncomputable def logTerm : EMLTerm  := .eml .one (.eml (.eml .one .var) .one)

/-- x - log(x) for x > 0 -/
noncomputable def xMinusLogTerm : EMLTerm  := .eml logTerm .var

/-- log(x - log(x)) for x > 0 -/
noncomputable def logXMinusLogTerm : EMLTerm  := .eml .one (.eml (.eml .one xMinusLogTerm)
 ↪  .one)

/-- -log(x) for x > 0 -/
noncomputable def negLogTerm : EMLTerm  := .eml logXMinusLogTerm (.eml .var .one)

/-- 1/x for x > 0 -/
noncomputable def invTerm : EMLTerm  := .eml negLogTerm .one

/-
Helper: x - log(x) > 0 for x > 0
-/
lemma sub_log_pos {x :  } (hx : 0 < x) : 0 < x - Real.log x := by
  linarith [ Real.log_le_sub_one_of_pos hx ]

-- Step 1: logTerm evaluates to log(x)
lemma eval_logTerm {x :  } (_hx : 0 < x) :
    EMLTerm.eval x logTerm = Real.log x := by
  simp only [logTerm, EMLTerm.eval, Real.log_one, sub_zero, Real.log_exp]
  ring

-- Step 2: xMinusLogTerm evaluates to x - log(x)
lemma eval_xMinusLogTerm {x :  } (hx : 0 < x) :
    EMLTerm.eval x xMinusLogTerm = x - Real.log x := by
  simp only [xMinusLogTerm, EMLTerm.eval, eval_logTerm hx, Real.exp_log hx]

-- Step 3: logXMinusLogTerm evaluates to log(x - log(x))
lemma eval_logXMinusLogTerm {x :  } (hx : 0 < x) :
    EMLTerm.eval x logXMinusLogTerm = Real.log (x - Real.log x) := by
  simp only [logXMinusLogTerm, EMLTerm.eval, eval_xMinusLogTerm hx, Real.log_one, sub_zero,
    Real.log_exp]
  ring

-- Step 4: negLogTerm evaluates to -log(x)
lemma eval_negLogTerm {x :  } (hx : 0 < x) :
    EMLTerm.eval x negLogTerm = -Real.log x := by
  simp only [negLogTerm, EMLTerm.eval, eval_logXMinusLogTerm hx,
    Real.exp_log (sub_log_pos hx), Real.log_one, sub_zero, Real.log_exp]
  ring

-- Step 5: invTerm evaluates to 1/x
```

```
lemma eval_invTerm {x :  } (hx : 0 < x) :
    EMLTerm .eval x invTerm = 1 / x := by
  simp only [invTerm, EMLTerm .eval, eval_negLogTerm hx, Real.log_one, sub_zero]
  rw [Real.exp_neg, Real.exp_log hx, one_div]

theorem emlterm1_for_inv_x_pos :
     t : EMLTerm ,   x :  , 0 < x → EMLTerm .eval x t = 1 / x := by
  exact  invTerm, fun x hx => eval_invTerm hx

end EML
```

038_emlterm_for_sq_x EMLTerm realising x^2 (for $x > 0$)

Paper section: §3 Results, EML expression catalog (x^2, K=75) • *Status:* complete • *Difficulty:* 5/5

x^2: K = 75 (compiler) / K = 17 (direct search).

There exists a parameterised EML term of size 75 (or 17 direct-search) whose evaluation equals x^2 for every x. Existential.

```
import Mathlib

namespace EML

inductive EMLTerm  : Type
  | one : EMLTerm
  | var : EMLTerm
  | eml : EMLTerm  → EMLTerm  → EMLTerm
  deriving Repr

noncomputable def EMLTerm .eval (x :  ) : EMLTerm  →
  | .one => 1
  | .var => x
  | .eml t u => Real.exp (EMLTerm .eval x t) - Real.log (EMLTerm .eval x u)

-- Building blocks
private def zeroTerm : EMLTerm  := .eml .one (.eml (.eml .one .one) .one)
private def logTerm : EMLTerm  := .eml zeroTerm (.eml (.eml zeroTerm .var) .one)
private def xMinusLogTerm : EMLTerm  := .eml logTerm .var
private def logXMinusLogTerm : EMLTerm  :=
  .eml zeroTerm (.eml (.eml zeroTerm xMinusLogTerm) .one)
private def xMinus2LogTerm : EMLTerm  :=
  .eml logXMinusLogTerm (.eml logTerm .one)
private def twoLogTerm : EMLTerm  :=
  .eml logTerm (.eml xMinus2LogTerm .one)
private def sqTerm : EMLTerm  := .eml twoLogTerm .one

/-
Helper: x - log x > 0 for x > 0
-/
private lemma x_minus_log_pos (x :  ) (hx : 0 < x) : 0 < x - Real.log x := by
  linarith [ Real.log_le_sub_one_of_pos hx ]

/-
Step-by-step evaluation lemmas
```

```
-/
private lemma eval_zeroTerm (x :  ) : zeroTerm.eval x = 0 := by
  simp [zeroTerm, EMLTerm .eval]

private lemma eval_logTerm (x :  ) (_hx : 0 < x) : logTerm.eval x = Real.log x := by
  unfold logTerm; simp +decide [ *, EMLTerm .eval ] ;

private lemma eval_xMinusLogTerm (x :  ) (hx : 0 < x) :
    xMinusLogTerm.eval x = x - Real.log x := by
      convert congr_arg  ( · - · ) ( Real.exp_log hx ) rfl using 1;
      convert congr_arg  ( · - · ) ( congr_arg Real.exp ( eval_logTerm x hx ) ) rfl using 1

private lemma eval_logXMinusLogTerm (x :  ) (hx : 0 < x) :
    logXMinusLogTerm.eval x = Real.log (x - Real.log x) := by
      unfold logXMinusLogTerm;
      -- We'll use the fact that $zeroTerm.eval x = 0$ and $xMinusLogTerm.eval x = x - \log
↪  x$.
      have h_eval : zeroTerm.eval x = 0   xMinusLogTerm.eval x = x - Real.log x := by
        exact   eval_zeroTerm x, eval_xMinusLogTerm x hx
      simp [h_eval, EMLTerm .eval]

private lemma eval_xMinus2LogTerm (x :  ) (hx : 0 < x) :
    xMinus2LogTerm.eval x = x - 2 * Real.log x := by
      convert congr_arg  ( · - · ) ( Real.exp_log ( x_minus_log_pos x hx ) ) ( Real.log_exp (
↪  Real.log x ) ) using 1;
      · rw [ show xMinus2LogTerm = .eml logXMinusLogTerm (.eml logTerm .one) from rfl, show
↪  logXMinusLogTerm = .eml zeroTerm (.eml (.eml zeroTerm xMinusLogTerm) .one) from rfl, show
↪  logTerm = .eml zeroTerm (.eml (.eml zeroTerm .var) .one) from rfl, show zeroTerm = .eml
↪  .one (.eml (.eml .one .one) .one) from rfl ] ; simp +decide [ EMLTerm .eval ] ;
        rw [ eval_xMinusLogTerm x hx ];
      · ring

private lemma eval_twoLogTerm (x :  ) (hx : 0 < x) :
    twoLogTerm.eval x = 2 * Real.log x := by
      unfold twoLogTerm;
      -- Apply the definitions of `logTerm` and `xMinus2LogTerm` to simplify the expression.
      simp [logTerm, xMinus2LogTerm, EMLTerm .eval];
      rw [ eval_logXMinusLogTerm x hx, Real.exp_log ( x_minus_log_pos x hx ) ] ; ring;
      rw [ Real.exp_log hx, sub_self, zero_add ]

private lemma eval_sqTerm (x :  ) (hx : 0 < x) :
    sqTerm.eval x = x ^ 2 := by
      -- Simplify $sqTerm$ using the results of $twoLogTerm$ and basic properties of
↪  exponentiation.
      have h_exp_simplified :
          Real.exp (2 * Real.log x) = x ^ 2 := by
            rw [ mul_comm, Real.exp_mul, Real.exp_log ] <;> norm_cast;
      convert h_exp_simplified using 1
      unfold EMLTerm .eval
      simp [sqTerm];
      rw [ eval_twoLogTerm x hx, show EMLTerm .eval x EMLTerm .one = 1 from by rfl,
↪  Real.log_one, sub_zero ]

theorem emlterm1_for_sq_x_pos :
     t : EMLTerm ,   x :  , 0 < x → EMLTerm .eval x t = x ^ 2 := by
  exact  sqTerm, eval_sqTerm
```

```
end EML
```

039_emlterm_for_sqrt_x EMLTerm realising √x (for x > 1)

Paper section: §3 Results, EML expression catalog (√x, K=139) • *Status:* complete • *Difficulty:* 5/5

√x: K = 139 (compiler) / K > 43 (direct search).

There exists a parameterised EML term of size 139 whose evaluation equals √x for x 0. PROBABLE PERMANENT SORRY: 139-node literal tree beyond the manual-transcription budget.

```
import Mathlib

namespace EML

inductive EMLTerm  : Type
  | one : EMLTerm
  | var : EMLTerm
  | eml : EMLTerm  → EMLTerm  → EMLTerm
  deriving Repr

noncomputable def EMLTerm .eval (x :  ) : EMLTerm  →
  | .one => 1
  | .var => x
  | .eml t u => Real.exp (EMLTerm .eval x t) - Real.log (EMLTerm .eval x u)

/-- Term that evaluates to `Real.log (eval x T)` for any `T` (unconditionally). -/
def mkLOG (T : EMLTerm ) : EMLTerm  := .eml .one (.eml (.eml .one T) .one)

/-- Term that evaluates to `Real.exp (eval x T)` for any `T` (unconditionally). -/
def mkEXP (T : EMLTerm ) : EMLTerm  := .eml T .one

/-- Term that evaluates to `eval x A - eval x B` when `eval x A > 0`. -/
def mkSUB (A B : EMLTerm ) : EMLTerm  := .eml (mkLOG A) (mkEXP B)

--
-- Evaluation lemmas for helpers
--

lemma eval_mkEXP (x :  ) (T : EMLTerm ) :
    EMLTerm .eval x (mkEXP T) = Real.exp (EMLTerm .eval x T) := by
  simp [mkEXP, EMLTerm .eval, Real.log_one]

/-
`mkLOG T` evaluates to `log(eval T)` unconditionally.
    Proof: eval = exp(1) - log(exp(exp(1) - log(eval T)) - log(1))
         = e - log(exp(e - log(eval T)))  (log 1 = 0)
         = e - (e - log(eval T))          (log   exp = id)
         = log(eval T)                    (ring)
-/
lemma eval_mkLOG (x :  ) (T : EMLTerm ) :
    EMLTerm .eval x (mkLOG T) = Real.log (EMLTerm .eval x T) := by
  -- By definition of $eval$, we know that $eval x (mkLOG T) = exp(1) - log(exp(exp(1) -
↪  log(eval T)) - log(1))$.
```

```
  have h_eval_mkLOG : EMLTerm.eval x (mkLOG T) = Real.exp 1 - Real.log (Real.exp (Real.exp 1
↪  - Real.log (EMLTerm.eval x T)) - Real.log 1) := by
    rfl;
  by_cases h : EMLTerm.eval x T = 0 <;> simp_all +decide [ Real.exp_ne_zero, sub_eq_add_neg
↪  ]

/-
`mkSUB A B` evaluates to `eval A - eval B` when `eval A > 0`.
    Proof: uses `eval_mkLOG`, `eval_mkEXP`, `exp(log(a)) = a` for `a > 0`,
    and `log(exp(b)) = b`.
-/
lemma eval_mkSUB (x : ) (A B : EMLTerm) (hA : 0 < EMLTerm.eval x A) :
    EMLTerm.eval x (mkSUB A B) = EMLTerm.eval x A - EMLTerm.eval x B := by
  unfold mkSUB; simp +decide [ *, EMLTerm.eval ] ; ring;
  rw [ eval_mkLOG, eval_mkEXP, Real.exp_log hA, Real.log_exp ]

--
-- Constant terms and evaluation lemmas
--

def E_term : EMLTerm := .eml .one .one

def EM1_term : EMLTerm := .eml .one E_term

def EM2_term : EMLTerm := mkSUB EM1_term .one

def TWO_term : EMLTerm := mkSUB E_term EM2_term

lemma eval_E (x : ) : EMLTerm.eval x E_term = Real.exp 1 := by
  simp [E_term, EMLTerm.eval, Real.log_one]

lemma eval_EM1 (x : ) : EMLTerm.eval x EM1_term = Real.exp 1 - 1 := by
  simp [EM1_term, E_term, EMLTerm.eval, Real.log_one, Real.log_exp]

lemma EM1_pos : (0 : ) < Real.exp 1 - 1 := by
  norm_num [ Real.exp_pos ]

lemma eval_EM2 (x : ) : EMLTerm.eval x EM2_term = Real.exp 1 - 2 := by
  rw [ show EM2_term = mkSUB EM1_term .one from rfl, eval_mkSUB ];
  · linarith [ eval_EM1 x, show EMLTerm.eval x EMLTerm.one = 1 from by rfl ];
  · exact eval_EM1 x  EM1_pos

lemma eval_TWO (x : ) : EMLTerm.eval x TWO_term = 2 := by
  rw [ show TWO_term = mkSUB E_term EM2_term from rfl, eval_mkSUB ];
  · rw [ eval_E, eval_EM2 ] ; ring;
  · exact eval_E x  Real.exp_pos _

--
-- ONE_PLUS_LOG2_term: evaluates to 1 + log 2
--

/-- `eml(one, exp(sub(EM1, log(TWO))))` evaluates to `1 + log 2`.
    Proof: eval = exp(1) - log(exp((e-1) - log(2)))
         = e - ((e-1) - log(2))
         = 1 + log(2) -/
def ONE_PLUS_LOG2_term : EMLTerm :=
```

```
  .eml .one (mkEXP (mkSUB EM1_term (mkLOG TWO_term)))

lemma eval_ONE_PLUS_LOG2 (x : ) :
    EMLTerm .eval x ONE_PLUS_LOG2_term = 1 + Real.log 2 := by
  unfold ONE_PLUS_LOG2_term;
  rw [ show EMLTerm .eval x ( EMLTerm .one.eml ( mkEXP ( mkSUB EM1_term ( mkLOG TWO_term ) ) )
↪  ) = Real.exp ( EMLTerm .eval x EMLTerm .one ) - Real.log ( EMLTerm .eval x ( mkEXP ( mkSUB
↪  EM1_term ( mkLOG TWO_term ) ) ) ) by rfl ] ; norm_num [ eval_mkEXP, eval_mkLOG,
↪  eval_mkSUB, eval_EM1, eval_TWO ] ; ring;
  rw [ show EMLTerm .eval x EMLTerm .one = 1 by rfl ] ; ring

lemma one_plus_log_two_pos : (0 : ) < 1 + Real.log 2 := by
  positivity

--
-- Variable-dependent terms
--

/-- Evaluates to `(1 + log 2) - log(log x)` for `x > 1`.
    Uses `eml(mkLOG(ONE_PLUS_LOG2_term), mkLOG(var))`, which computes
    `exp(log(1+log 2)) - log(log x) = (1+log 2) - log(log x)`. -/
def one_plus_c_term : EMLTerm  :=
  .eml (mkLOG ONE_PLUS_LOG2_term) (mkLOG .var)

lemma eval_one_plus_c (x : ) (hx : 1 < x) :
    EMLTerm .eval x one_plus_c_term =
    (1 + Real.log 2) - Real.log (Real.log x) := by
  -- Apply the definition of `eval` for `eml` terms.
  simp [one_plus_c_term, EMLTerm .eval];
  rw [ eval_mkLOG, eval_ONE_PLUS_LOG2, Real.exp_log one_plus_log_two_pos, eval_mkLOG,
↪  EMLTerm .eval ]

/-- The sqrt term: `mkEXP(mkEXP(mkSUB(one, one_plus_c_term)))`.
    For `x > 1`:
    eval = exp(exp(1 - ((1+log 2) - log(log x))))
         = exp(exp(log(log x) - log 2))
         = exp(exp(log(log x / 2)))        (log_div)
         = exp(log x / 2)                  (exp_log, log x / 2 > 0)
         = √x                              (exp(log x / 2) = x^(1/2) = √x) -/
def sqrt_term  : EMLTerm  := mkEXP (mkEXP (mkSUB .one one_plus_c_term))

lemma eval_sqrt  (x : ) (hx : 1 < x) :
    EMLTerm .eval x sqrt_term  = Real.sqrt x := by
  convert eval_mkEXP x ( mkEXP ( mkSUB .one one_plus_c_term ) ) using 1;
  rw [ eval_mkEXP, eval_mkSUB ];
  · rw [ eval_one_plus_c x hx ] ; ring;
    rw [ Real.sqrt_eq_rpow, Real.rpow_def_of_pos ( by positivity ) ] ; norm_num [
↪  EMLTerm .eval ] ; ring;
    rw [ Real.exp_add, Real.exp_neg, Real.exp_log, Real.exp_log ] <;> ring <;> norm_num [
↪  Real.log_pos hx ];
  · exact zero_lt_one

--
-- Main theorem
--
```

```
theorem emlterm1_for_sqrt_x_gt_one :
     t : EMLTerm ,   x :  , 1 < x → EMLTerm .eval x t = Real.sqrt x :=
  sqrt_term , fun x hx => eval_sqrt  x hx

end EML
```

040_emlterm_for_add_xy EMLTerm realising x + y

Paper section: §3 Results, EML expression catalog (x + y, K=27) • *Status:* complete • *Difficulty:* 5/5

x + y: K = 27 (compiler) / K = 19 (direct search).

There exists a two-variable EML term of size 27 whose evaluation at (x,y) equals x + y. Requires a 2-variable grammar EMLTerm (with .varX, .varY leaves) and an evaluation eval : → → EMLTerm → .

```
import Mathlib

namespace EML

/-- Two-variable EML term grammar. -/
inductive EMLTerm  : Type
  | one : EMLTerm
  | varX : EMLTerm
  | varY : EMLTerm
  | eml : EMLTerm  → EMLTerm  → EMLTerm
  deriving Repr

/-- Evaluation of a two-variable EML term at (x, y). -/
noncomputable def EMLTerm .eval (x y :  ) : EMLTerm  →
  | .one => 1
  | .varX => x
  | .varY => y
  | .eml t u => Real.exp (EMLTerm .eval x y t) - Real.log (EMLTerm .eval x y u)

/-
exp(x) - x is always positive.
-/
lemma exp_sub_self_pos (x :  ) : 0 < Real.exp x - x := by
  linarith [ Real.add_one_le_exp x ]

theorem emlterm2_for_add :
     t : EMLTerm ,   x y :  , EMLTerm .eval x y t = x + y := by
  refine  .eml
    (.eml .one (.eml (.eml .one (.eml .varX .one)) .one))
    (.eml
      (.eml (.eml .one (.eml (.eml .one (.eml .varX (.eml .varX .one))) .one))
            (.eml .varY .one))
      .one), ?_
  intro x y
  simp only [EMLTerm .eval, Real.log_one, sub_zero, Real.log_exp]
  have h1 : Real.exp 1 - (Real.exp 1 - x) = x := by ring
  have h2 : Real.exp 1 - (Real.exp 1 - Real.log (Real.exp x - x)) =
    Real.log (Real.exp x - x) := by ring
  rw [h1, h2, Real.exp_log (exp_sub_self_pos x)]
```

```
  ring

end EML
```

041_emlterm_for_mul_xy EMLTerm realising x · y

Paper section: §3 Results, EML expression catalog (x × y, K=41) • *Status:* complete • *Difficulty:* 5/5

x × y: K = 41 (compiler) / K = 17 (direct search).

There exists a two-variable EML term of size 41 whose evaluation equals x · y for x, y > 0 (Identity 1). Existential.

```
import Mathlib

namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | varX : EMLTerm
  | varY : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

noncomputable def EMLTerm.eval (x y : ℝ) : EMLTerm → ℝ
  | .one => 1
  | .varX => x
  | .varY => y
  | .eml t u => Real.exp (EMLTerm.eval x y t) - Real.log (EMLTerm.eval x y u)

/-
For x > 0, x - log x > 0
-/
lemma sub_log_pos {x : ℝ} (hx : 0 < x) : 0 < x - Real.log x := by
  linarith [ Real.log_le_sub_one_of_pos hx ]

theorem emlterm2_for_mul :
    ∀ t : EMLTerm, ∀ x y : ℝ, 0 < x → 0 < y → EMLTerm.eval x y t = x * y := by
  refine ?_, fun x y hx hy => ?_
  · exact .eml (.eml (.eml .one (.eml (.eml .one .varX) .one))
      (.eml (.eml (.eml .one (.eml (.eml .one
        (.eml (.eml .one (.eml (.eml .one .varX) .one))
          (.eml (.eml .one (.eml (.eml .one .varX) .one)) .one))) .one)) .varY) .one)) .one
  · simp only [EMLTerm.eval, Real.log_one, sub_zero, Real.log_exp]
    -- Goal has e - (e - log x) patterns where e = exp 1
    set e := Real.exp 1
    -- Step 1: simplify e - (e - log x) to log x
    have h1 : e - (e - Real.log x) = Real.log x := by ring
    rw [h1]
    -- Step 2: exp(log x) = x
    rw [Real.exp_log hx]
    -- Step 3: simplify e - (e - log(x - log x)) to log(x - log x)
    have h3 : e - (e - Real.log (x - Real.log x)) = Real.log (x - Real.log x) := by ring
    rw [h3]
    -- Step 4: exp(log(x - log x)) = x - log x
```

```
    rw [Real.exp_log (sub_log_pos hx)]
    -- Step 5 & 6: exp(x - (x - log x - log y)) = exp(log x + log y) = x * y
    have h5 : x - (x - Real.log x - Real.log y) = Real.log x + Real.log y := by ring
    rw [h5, Real.exp_add, Real.exp_log hx, Real.exp_log hy]

end EML
```

042_emlterm_for_pow_xy EMLTerm realising x^y (for 0 < x and 0 < y)

Paper section: §3 Results, EML expression catalog (x^y, K=49) • *Status:* complete • *Difficulty:* 5/5

x^y: K = 49 (compiler) / K = 25 (direct search).

There exists a two-variable EML term of size 49 (or 25 direct-search) whose evaluation equals x^y = exp(y · ln x) for x > 0. Existential.

```
import Mathlib

namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | varX : EMLTerm
  | varY : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

noncomputable def EMLTerm.eval (x y : ℝ) : EMLTerm → ℝ
  | .one => 1
  | .varX => x
  | .varY => y
  | .eml t u => Real.exp (EMLTerm.eval x y t) - Real.log (EMLTerm.eval x y u)

-- Helper definitions for building the term
private def Z : EMLTerm := .eml .one (.eml (.eml .one .one) .one)
private def LOG (a : EMLTerm) : EMLTerm := .eml Z (.eml (.eml Z a) .one)
private def NEG_LOG (v : EMLTerm) (raw : EMLTerm) : EMLTerm :=
  .eml (LOG (.eml v raw)) (.eml raw .one)

/-- EML term that computes x^y for x > 0, y > 0.
Key identity: y * log(x) = y * (1/x + log(x)) - y/x,
where both 1/x + log(x) > 0 and 1/x > 0 for x > 0. -/
noncomputable def pow_term : EMLTerm :=
  let logx := LOG .varX
  let logy := LOG .varY
  let neg_logx := NEG_LOG logx .varX
  let neg_logy := NEG_LOG logy .varY
  let inv_y_plus_logy := EMLTerm.eml neg_logy (.eml neg_logy .one)
  let log_inv_y_plus_logy := LOG inv_y_plus_logy
  let inv_x_plus_logx := EMLTerm.eml neg_logx (.eml neg_logx .one)
  let log_inv_x_plus_logx := LOG inv_x_plus_logx
  let A_arg := EMLTerm.eml log_inv_y_plus_logy
    (.eml (.eml neg_logy (.eml log_inv_x_plus_logx .one)) .one)
  let B_arg := EMLTerm.eml log_inv_y_plus_logy
    (.eml (.eml neg_logy (.eml neg_logx .one)) .one)
```

```
  let A := EMLTerm.eml A_arg .one
  let B := EMLTerm.eml B_arg .one
  let y_logx := EMLTerm.eml (LOG A) (.eml B .one)
  EMLTerm.eml y_logx .one

/-
Auxiliary lemmas
-/
private lemma inv_add_log_pos {a : } (ha : 0 < a) : 0 < a⁻¹ + Real.log a := by
  nlinarith [ inv_pos.2 ha, mul_inv_cancel ha.ne', Real.log_inv a
↪  Real.log_le_sub_one_of_pos ( inv_pos.2 ha ) ]

private lemma eval_Z (x y : ) : EMLTerm.eval x y Z = 0 := by
  unfold Z;
  -- By definition of $Z$, we know that $Z = \exp(1) - \log(\exp(\exp(1) - \log(1)))$.
  simp [EMLTerm.eval]

private lemma eval_LOG (x y : ) (a : EMLTerm) (ha : 0 < EMLTerm.eval x y a) :
    EMLTerm.eval x y (LOG a) = Real.log (EMLTerm.eval x y a) := by
  unfold LOG;
  -- By definition of `LOG`, we know that `EMLTerm.eval x y (Z.eml ((Z.eml a).eml
↪  EMLTerm.one)) = Real.log (EMLTerm.eval x y a)`.
  simp [EMLTerm.eval]

private lemma eval_NEG_LOG (x y : ) (v raw : EMLTerm)
    (hraw : 0 < EMLTerm.eval x y raw)
    (hv : EMLTerm.eval x y v = Real.log (EMLTerm.eval x y raw))
    (_hd : 0 < EMLTerm.eval x y raw - EMLTerm.eval x y v) :
    EMLTerm.eval x y (NEG_LOG v raw) = -(EMLTerm.eval x y v) := by
  unfold NEG_LOG;
  unfold LOG; norm_num [ EMLTerm.eval ] ;
  rw [ Real.exp_log ] <;> norm_num [ hv ] ; linarith [ Real.exp_log hraw ];
  linarith [ Real.add_one_le_exp ( Real.log ( EMLTerm.eval x y raw ) ) ]

private lemma eval_pow_term_eq (x y : ) (hx : 0 < x) (hy : 0 < y) :
    EMLTerm.eval x y pow_term = Real.exp (y * Real.log x) := by
  -- Let's simplify the expression step by step.
  have h1 : EMLTerm.eval x y (LOG .varX) = Real.log x := by
    exact eval_LOG x y _ hx
  have h2 : EMLTerm.eval x y (LOG .varY) = Real.log y := by
    exact eval_LOG x y _ hy
  have h3 : EMLTerm.eval x y (NEG_LOG (LOG .varX) .varX) = -Real.log x := by
    rw [ ← h1 ];
    apply_rules [ eval_NEG_LOG ];
    exact sub_pos_of_lt ( by linarith [ Real.log_le_sub_one_of_pos hx, show EMLTerm.eval x y
↪  EMLTerm.varX = x from rfl ] )
  have h4 : EMLTerm.eval x y (NEG_LOG (LOG .varY) .varY) = -Real.log y := by
    rw [ ← h2 ];
    apply_rules [ eval_NEG_LOG ];
    exact sub_pos_of_lt ( by linarith [ Real.log_le_sub_one_of_pos hy, show EMLTerm.eval x y
↪  EMLTerm.varY = y from rfl ] );
  unfold pow_term; simp +decide [ *, EMLTerm.eval ] ; ring;
  simp +decide [ *, EMLTerm.eval, LOG ] at *;
  norm_num [ Real.exp_add, Real.exp_sub, Real.exp_neg, Real.exp_log hx, Real.exp_log hy ] ;
↪  ring;
  rw [ Real.exp_log ( by linarith [ inv_pos.2 hy, Real.log_inv y  Real.log_le_sub_one_of_pos
↪  ( inv_pos.2 hy ) ] ), Real.exp_log ( by linarith [ inv_pos.2 hx, Real.log_inv x
↪  Real.log_le_sub_one_of_pos ( inv_pos.2 hx ) ] ) ] ; norm_num [ Real.exp_ne_zero, hx.ne',
↪  hy.ne' ] ; ring;
```

```
  norm_num [ Real.exp_add, Real.exp_log hy, mul_assoc, mul_comm, mul_left_comm, ne_of_gt (
↪  Real.exp_pos _ ) ]

theorem emlterm2_for_pow_pos :
     t : EMLTerm ,   x y :  , 0 < x → 0 < y → EMLTerm .eval x y t = x ^ y := by
  exact  pow_term, fun x y hx hy => by
    rw [eval_pow_term_eq x y hx hy]
    rw [Real.rpow_def_of_pos hx]
    ring_nf

end EML
```

043_master_formula_param_count Master-formula parameter count at level n

Paper section: §4.3 Master formula - symbolic regression • *Status:* complete • *Difficulty:* 2/5

Level-n EML master formula has $5 \times 2\hat{}n - 6$ parameters total.

The level-n master formula has $5 \cdot 2\hat{}n - 6$ parameters. We define parametrCount n := $5 \cdot 2\hat{}n - 6$ and check small values (n=1: 4, n=2: 14, n=3: 34).

```
import Mathlib

namespace EML

/-- Total parameter count of the level-n EML master formula:
`5 · 2^n - 6` (Section 4.3). -/
def masterParamCount (n :  ) :   := 5 * 2 ^ n - 6

example : masterParamCount 1 = 4 := by native_decide
example : masterParamCount 2 = 14 := by native_decide
example : masterParamCount 3 = 34 := by native_decide

end EML
```

044_emlterm_count_catalan Count of EMLTerms equals the Catalan number

Paper section: §4.2 Elementary functions as binary trees ('Catalan structures') • *Status:* complete • *Difficulty:* 4/5

Context-free language; isomorphic to full binary trees / Catalan structures.

The number of full binary trees with n leaves is the Catalan number C_{n−1}. By induction, the count of EMLTerms of size 2k+1 is C_k. Mathlib has `Nat.catalan`.

```
import Mathlib

namespace EML

inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr, DecidableEq

def EMLTerm.size : EMLTerm →
  | .one => 1
```

```
  | .eml t u => 1 + EMLTerm.size t + EMLTerm.size u

lemma EMLTerm.size_odd :  t : EMLTerm,  k, t.size = 2 * k + 1
  | .one =>  0, by simp [EMLTerm.size]
  | .eml a b => by
    obtain  i, hi  := EMLTerm.size_odd a
    obtain  j, hj  := EMLTerm.size_odd b
    exact  i + j + 1, by simp only [EMLTerm.size, hi, hj]; omega

lemma EMLTerm.size_pos :  t : EMLTerm, 0 < t.size
  | .one => by simp [EMLTerm.size]
  | .eml a b => by simp [EMLTerm.size]

def termsOfInternalNodes :  → Finset EMLTerm
  | 0 => {.one}
  | k + 1 =>
    ((Finset.range (k + 1)).attach).biUnion fun  i, hi  =>
      have h1 : i < k + 1 := Finset.mem_range.mp hi
      have h2 : k - i < k + 1 := by omega
      ((termsOfInternalNodes i) ×  (termsOfInternalNodes (k - i))).image
        fun p => EMLTerm.eml p.1 p.2
termination_by k => k
decreasing_by all_goals omega

@[simp] lemma termsOfInternalNodes_zero : termsOfInternalNodes 0 = {.one} := by
  simp [termsOfInternalNodes]

lemma size_eq_of_mem_termsOfInternalNodes (k :  ) (t : EMLTerm)
    (ht : t  termsOfInternalNodes k) : EMLTerm.size t = 2 * k + 1 := by
  induction' k using Nat.strong_induction_on with k ih generalizing t
  unfold termsOfInternalNodes at ht
  rcases k with ( _ | k ) <;> simp_all +decide
  rcases ht with  a, ha, b, c,  hb, hc  , rfl
  simp +arith +decide [ *, EMLTerm.size ]
  grind

lemma mem_termsOfInternalNodes_of_size (k :  ) (t : EMLTerm)
    (ht : t.size = 2 * k + 1) : t  termsOfInternalNodes k := by
  induction' k using Nat.strong_induction_on with k ih generalizing t
  rcases k with ( _ | k ) <;> simp_all +decide
  · cases t
    · rfl
    · exact absurd ht (by erw [show (_ : EMLTerm).size = 1 + (_ : EMLTerm).size + (_ :
↪  EMLTerm).size from rfl]; linarith [EMLTerm.size_pos <_>, EMLTerm.size_pos <_>])
  · rcases t with ( _ |  a, b  )
    · cases ht
    · obtain  i, hi  :  i, i  k  a.size = 2 * i + 1  b.size = 2 * (k - i) + 1 := by
        obtain  i, hi  := EMLTerm.size_odd a
        obtain  j, hj  := EMLTerm.size_odd b
        use i
        simp_all +decide [ EMLTerm.size ]
        omega
      unfold termsOfInternalNodes; aesop

/-
Helper: eml is injective as a function on pairs
```

```
-/
lemma eml_pair_injective : Function.Injective (fun p : EMLTerm × EMLTerm => EMLTerm.eml p.1
↪  p.2) := by
  -- To prove injectivity, assume that eml p1 p2 = eml q1 q2. By the definition of eml, this
↪  implies that p1 = q1 and p2 = q2.
  intro p1 p2 h_eq
  simp [EMLTerm.eml] at h_eq
  aesop

/-
Helper: the image of eml on a product has card = card A * card B
-/
lemma card_eml_image (A B : Finset EMLTerm) :
    ((A ×  B).image fun p => EMLTerm.eml p.1 p.2).card = A.card * B.card := by
  rw [ Finset.card_image_of_injective ] <;> norm_num [ Function.Injective, eml_pair_injective
↪  ]

/-
Helper: disjointness of images for different i
-/
lemma eml_images_pairwise_disjoint (k :  ) :
    Set.PairwiseDisjoint (↑(Finset.range (k + 1)).attach)
      (fun (x : { x // x   Finset.range (k + 1) }) =>
        ((termsOfInternalNodes x.1) ×  (termsOfInternalNodes (k - x.1))).image
          fun p => EMLTerm.eml p.1 p.2) := by
  intro x hx y hy hxy; simp_all +decide [ Finset.disjoint_left ] ;
  rintro a u v hu hv rfl w z hw hz; contrapose! hxy;
  have := size_eq_of_mem_termsOfInternalNodes x.val u hu; have :=
↪  size_eq_of_mem_termsOfInternalNodes y.val w hw; aesop;

/-
The cardinality proof using the helpers
-/
lemma card_termsOfInternalNodes (k :  ) :
    (termsOfInternalNodes k).card = catalan k := by
  induction' k using Nat.case_strong_induction_on with k ih;
  · -- The base case when $k = 0$ follows directly from the definition of
↪  `termsOfInternalNodes`.
    simp [termsOfInternalNodes];
  · have h_card : (termsOfInternalNodes (k + 1)).card =   i   Finset.range (k + 1),
↪  (termsOfInternalNodes i).card * (termsOfInternalNodes (k - i)).card := by
      rw [ termsOfInternalNodes, Finset.card_biUnion ];
      · refine' Finset.sum_bij ( fun x hx => x.val ) _ _ _ _ <;> simp +decide [
↪  card_eml_image ];
      · convert eml_images_pairwise_disjoint k using 1;
    simp_all +decide [ catalan_succ' ];
    rw [ Finset.Nat.sum_antidiagonal_eq_sum_range_succ fun i j => catalan i * catalan j ];
    exact Finset.sum_congr rfl fun x hx => by rw [ ih x ( Finset.mem_range_succ_iff.mp hx ) ]
↪  ;

/-- Number of EML terms of size `2k + 1` equals the Catalan number `C `. -/
theorem emlterm_count_catalan (k :  ) :
     (S : Finset EMLTerm), ( t   S, EMLTerm.size t = 2 * k + 1)
     ( t : EMLTerm, EMLTerm.size t = 2 * k + 1 → t   S)
     S.card = catalan k :=
  termsOfInternalNodes k,
```

```
    fun t ht => size_eq_of_mem_termsOfInternalNodes k t ht,
    fun t ht => mem_termsOfInternalNodes_of_size k t ht,
    card_termsOfInternalNodes k

end EML
```

045_main_completeness_stub Main completeness theorem — eleven-conjunct umbrella

Paper section: §3 Results, abstract claim of universality • *Status:* complete • *Difficulty:* 5/5

EML + 1 generates all standard scientific calculator operations.

Umbrella statement (final form): an eleven-conjunct existential covering each constructive sub-case (chunks 030, 031, 032, 033, 022, 036, 037, 038, 040, 041, 042). Excludes (034), i (035), √x (039) — those need the paper's Supplementary trees and remain permanent sorries.

```
import Mathlib

/-!
# Main completeness umbrella for the EML formalization (chunk 045).

This file is **self-contained**: it redefines the three EMLTerm shapes
(`EMLTerm`, `EMLTerm `, `EMLTerm `) and their `eval` functions, then
inlines the constructive witnesses harvested from chunks 030, 031, 032,
033, 022, 036, 037, 038, 040, 041, 042, and finally bundles them into a
single 11-conjunct existential.

Note:  (chunk 034), i (chunk 035), and √x (chunk 039) are **not** part of
this umbrella - their witnesses require the paper's Supplementary trees,
which are kept as permanent sorries elsewhere.
-/

namespace EML

/-! ## Term shapes -/

/-- Closed EML term (no variables). -/
inductive EMLTerm : Type
  | one : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

/-- Single-variable EML term. -/
inductive EMLTerm  : Type
  | one : EMLTerm
  | var : EMLTerm
  | eml : EMLTerm  → EMLTerm  → EMLTerm
  deriving Repr

/-- Two-variable EML term. -/
inductive EMLTerm  : Type
  | one : EMLTerm
  | varX : EMLTerm
```

```
  | varY : EMLTerm
  | eml : EMLTerm → EMLTerm → EMLTerm
  deriving Repr

noncomputable def EMLTerm.eval : EMLTerm →
  | .one => 1
  | .eml t u => Real.exp (EMLTerm.eval t) - Real.log (EMLTerm.eval u)

noncomputable def EMLTerm.eval (x : ) : EMLTerm →
  | .one => 1
  | .var => x
  | .eml t u => Real.exp (EMLTerm.eval x t) - Real.log (EMLTerm.eval x u)

noncomputable def EMLTerm.eval (x y : ) : EMLTerm →
  | .one => 1
  | .varX => x
  | .varY => y
  | .eml t u => Real.exp (EMLTerm.eval x y t) - Real.log (EMLTerm.eval x y u)

/-! ## Shared positivity helpers -/

private lemma exp_one_sub_one_pos : (0 : ) < Real.exp 1 - 1 := by
  linarith [Real.add_one_le_exp (1 : )]

private lemma exp_sub_self_pos (x : ) : 0 < Real.exp x - x := by
  linarith [Real.add_one_le_exp x]

private lemma sub_log_pos {x : } (hx : 0 < x) : 0 < x - Real.log x := by
  linarith [Real.log_le_sub_one_of_pos hx]

private lemma inv_add_log_pos {a : } (ha : 0 < a) : 0 < a⁻¹ + Real.log a := by
  nlinarith [inv_pos.2 ha, mul_inv_cancel ha.ne',
    Real.log_inv a  Real.log_le_sub_one_of_pos (inv_pos.2 ha)]

/-! ## Conjunct 1 (chunk 030): zero is EML-representable -/

private theorem c030_zero :  t : EMLTerm, EMLTerm.eval t = 0 := by
  refine .eml .one (.eml (.eml .one .one) .one), ?_
  simp [EMLTerm.eval, Real.log_one, sub_zero, Real.log_exp, sub_self]

/-! ## Conjunct 2 (chunk 031): -1 is EML-representable -/

private theorem c031_neg_one :  t : EMLTerm, EMLTerm.eval t = -1 := by
  refine .eml (.eml .one (.eml (.eml .one (.eml .one (.eml .one .one))) .one))
           (.eml (.eml .one .one) .one), ?_
  simp [EMLTerm.eval]
  rw [Real.exp_log] <;> linarith [Real.add_one_le_exp 1]

/-! ## Conjunct 3 (chunk 032): 2 is EML-representable -/

private theorem c032_two :  t : EMLTerm, EMLTerm.eval t = 2 := by
  set t2 : EMLTerm := .eml .one .one with ht2
  set t3 : EMLTerm := .eml .one t2 with ht3
  set t4 : EMLTerm := .eml .one t3 with ht4
  set t5 : EMLTerm := .eml t4 .one with ht5
  set t6 : EMLTerm := .eml .one t5 with ht6
```

```
  set t7 : EMLTerm := .eml t6 t2 with ht7
  set t8 : EMLTerm := .eml t7 .one with ht8
  refine  .eml .one t8, ?_
  have e2 : EMLTerm.eval t2 = Real.exp 1 := by
    simp [ht2, EMLTerm.eval, Real.log_one]
  have e3 : EMLTerm.eval t3 = Real.exp 1 - 1 := by
    simp [ht3, EMLTerm.eval, e2, Real.log_exp]
  have e4 : EMLTerm.eval t4 = Real.exp 1 - Real.log (Real.exp 1 - 1) := by
    simp [ht4, EMLTerm.eval, e3]
  have e5 : EMLTerm.eval t5 = Real.exp (Real.exp 1 - Real.log (Real.exp 1 - 1)) := by
    simp [ht5, EMLTerm.eval, e4, Real.log_one]
  have e6 : EMLTerm.eval t6 = Real.log (Real.exp 1 - 1) := by
    simp [ht6, EMLTerm.eval, e5, Real.log_exp]
  have e7 : EMLTerm.eval t7 = Real.exp 1 - 2 := by
    simp only [ht7, EMLTerm.eval, e6, e2]
    rw [Real.exp_log exp_one_sub_one_pos]
    linarith [Real.log_exp 1]
  have e8 : EMLTerm.eval t8 = Real.exp (Real.exp 1 - 2) := by
    simp [ht8, EMLTerm.eval, e7, Real.log_one]
  simp only [EMLTerm.eval, e8, Real.log_exp]
  ring

/-! ## Conjunct 4 (chunk 033): 1/2 is EML-representable -/

private theorem c033_half :   t : EMLTerm, EMLTerm.eval t = 1 / 2 := by
  set Z : EMLTerm := .eml .one (.eml (.eml .one .one) .one) with hZ
  let Lg : EMLTerm → EMLTerm := fun t => .eml Z (.eml (.eml Z t) .one)
  set e1 : EMLTerm := .eml .one (.eml .one .one) with he1
  set log_e1 : EMLTerm := Lg e1 with hle1
  set e2 : EMLTerm := .eml log_e1 (.eml .one .one) with he2
  set exp_e2 : EMLTerm := .eml e2 .one with hexpe2
  set two_t : EMLTerm := .eml .one exp_e2 with htwo_t
  set eml2 : EMLTerm := .eml .one two_t with heml2
  set log_eml2 : EMLTerm := Lg eml2 with hle2
  set neg_log2 : EMLTerm := .eml log_eml2 (.eml (.eml .one .one) .one) with hnl2
  set half_term : EMLTerm := .eml neg_log2 .one with hht
  refine  half_term, ?_
  have eval_Z : EMLTerm.eval Z = 0 := by
    simp [hZ, EMLTerm.eval, Real.log_one, Real.log_exp]
  have eval_Lg :   s : EMLTerm, 0 < EMLTerm.eval s →
      EMLTerm.eval (Lg s) = Real.log (EMLTerm.eval s) := by
    intro s _
    show EMLTerm.eval (.eml Z (.eml (.eml Z s) .one)) = _
    simp only [EMLTerm.eval, eval_Z, Real.exp_zero, Real.log_exp, Real.log_one, sub_zero]
    ring
  have eval_e1 : EMLTerm.eval e1 = Real.exp 1 - 1 := by
    simp [he1, EMLTerm.eval, Real.log_one, Real.log_exp]
  have eval_log_e1 : EMLTerm.eval log_e1 = Real.log (Real.exp 1 - 1) := by
    rw [hle1, eval_Lg e1 (by rw [eval_e1]; exact exp_one_sub_one_pos), eval_e1]
  have eval_e2 : EMLTerm.eval e2 = Real.exp 1 - 2 := by
    simp only [he2, EMLTerm.eval, eval_log_e1, Real.exp_log exp_one_sub_one_pos,
      Real.log_one, sub_zero, Real.log_exp]
    ring
  have eval_exp_e2 : EMLTerm.eval exp_e2 = Real.exp (Real.exp 1 - 2) := by
    simp only [hexpe2, EMLTerm.eval, eval_e2, Real.log_one, sub_zero]
  have eval_two : EMLTerm.eval two_t = 2 := by
```

```
    simp only [htwo_t, EMLTerm.eval, eval_exp_e2, Real.log_exp]; ring
  have eval_eml2 : EMLTerm.eval eml2 = Real.exp 1 - Real.log 2 := by
    simp only [heml2, EMLTerm.eval, eval_two]
  have log_two_le_one : Real.log 2  1 := by
    rw [show (1 :  ) = Real.log (Real.exp 1) from (Real.log_exp 1).symm]
    exact Real.log_le_log (by norm_num) (by linarith [Real.add_one_le_exp (1 :  )])
  have exp_one_sub_log_two_pos : (0 :  ) < Real.exp 1 - Real.log 2 := by
    linarith [exp_one_sub_one_pos, log_two_le_one]
  have eval_log_eml2 : EMLTerm.eval log_eml2 = Real.log (Real.exp 1 - Real.log 2) := by
    rw [hle2, eval_Lg eml2 (by rw [eval_eml2]; exact exp_one_sub_log_two_pos), eval_eml2]
  have eval_neg_log2 : EMLTerm.eval neg_log2 = -Real.log 2 := by
    simp only [hnl2, EMLTerm.eval, eval_log_eml2, Real.log_exp,
      Real.exp_log exp_one_sub_log_two_pos, Real.log_one, sub_zero]
    ring
  simp only [hht, EMLTerm.eval, eval_neg_log2, Real.log_one, sub_zero,
    Real.exp_neg, Real.exp_log (by norm_num : (0 :  ) < 2)]
  norm_num

/-! ## Conjunct 5 (chunk 022): e is EML-representable -/

private theorem c022_e :   t : EMLTerm, EMLTerm.eval t = Real.exp 1 := by
  refine  .eml .one .one, ?_
  simp [EMLTerm.eval, Real.log_one]

/-! ## Conjunct 6 (chunk 036): negation is EML-representable -/

private theorem c036_neg_x :
      t : EMLTerm ,   x :  , EMLTerm .eval x t = -x := by
  -- Witness: eml (eml one (eml (eml one w) one)) (eml expx one)
  --   where w = eml var (eml var one), expx = eml var one.
  refine  .eml
    (.eml .one (.eml (.eml .one (.eml .var (.eml .var .one))) .one))
    (.eml (.eml .var .one) .one), ?_
  intro x
  -- Step-by-step unfold via simp on EMLTerm .eval.
  show Real.exp (EMLTerm .eval x
        (.eml .one (.eml (.eml .one (.eml .var (.eml .var .one))) .one))) -
      Real.log (EMLTerm .eval x (.eml (.eml .var .one) .one)) = -x
  simp only [EMLTerm .eval, Real.log_one, sub_zero, Real.log_exp]
  -- Goal: exp(1 - log(exp(1 - log(exp x - x)))) - exp x = -x  (using log_exp/log_one
↪  rewrites)
  -- Actually after simp: exp 1 - (exp 1 - log(exp x - x)) appears and gets log_exp'd.
  rw [show Real.exp 1 - (Real.exp 1 - Real.log (Real.exp x - x)) =
        Real.log (Real.exp x - x) from by ring]
  rw [Real.exp_log (exp_sub_self_pos x)]
  ring

/-! ## Conjunct 7 (chunk 037): reciprocal (positive case) is EML-representable -/

private theorem c037_inv_x :
      t : EMLTerm ,   x :  , 0 < x → EMLTerm .eval x t = 1 / x := by
  set logTerm : EMLTerm  := .eml .one (.eml (.eml .one .var) .one) with hlogTerm
  set xMinusLogTerm : EMLTerm  := .eml logTerm .var with hxmlt
  set logXMinusLogTerm : EMLTerm  :=
    .eml .one (.eml (.eml .one xMinusLogTerm) .one) with hlxmlt
  set negLogTerm : EMLTerm  := .eml logXMinusLogTerm (.eml .var .one) with hnlt
```

```
  set invTerm : EMLTerm  := .eml negLogTerm .one with hinvT
  refine  invTerm, fun x hx => ?_
  have eval_logTerm : EMLTerm .eval x logTerm = Real.log x := by
    simp only [hlogTerm, EMLTerm .eval, Real.log_one, sub_zero, Real.log_exp]
    ring
  have eval_xMinusLogTerm : EMLTerm .eval x xMinusLogTerm = x - Real.log x := by
    simp only [hxmlt, EMLTerm .eval, eval_logTerm, Real.exp_log hx]
  have eval_logXMinusLogTerm :
      EMLTerm .eval x logXMinusLogTerm = Real.log (x - Real.log x) := by
    simp only [hlxmlt, EMLTerm .eval, eval_xMinusLogTerm,
      Real.log_one, sub_zero, Real.log_exp]
    ring
  have eval_negLogTerm : EMLTerm .eval x negLogTerm = -Real.log x := by
    simp only [hnlt, EMLTerm .eval, eval_logXMinusLogTerm,
      Real.exp_log (sub_log_pos hx), Real.log_one, sub_zero, Real.log_exp]
    ring
  simp only [hinvT, EMLTerm .eval, eval_negLogTerm, Real.log_one, sub_zero]
  rw [Real.exp_neg, Real.exp_log hx, one_div]

/-! ## Conjunct 8 (chunk 038): square (positive case) is EML-representable -/

private theorem c038_sq_x :
      t : EMLTerm ,   x :  , 0 < x → EMLTerm .eval x t = x ^ 2 := by
  set zeroT : EMLTerm  := .eml .one (.eml (.eml .one .one) .one) with hzeroT
  set logT : EMLTerm  := .eml zeroT (.eml (.eml zeroT .var) .one) with hlogT
  set xMinusLogT : EMLTerm  := .eml logT .var with hxml
  set logXMinusLogT : EMLTerm  :=
    .eml zeroT (.eml (.eml zeroT xMinusLogT) .one) with hlxml
  set xMinus2LogT : EMLTerm  := .eml logXMinusLogT (.eml logT .one) with hx2l
  set twoLogT : EMLTerm  := .eml logT (.eml xMinus2LogT .one) with htl
  set sqT : EMLTerm  := .eml twoLogT .one with hsqT
  refine  sqT, fun x hx => ?_
  have eval_zeroT : EMLTerm .eval x zeroT = 0 := by
    simp [hzeroT, EMLTerm .eval, Real.log_one, Real.log_exp]
  have eval_logT : EMLTerm .eval x logT = Real.log x := by
    simp only [hlogT, EMLTerm .eval, eval_zeroT, Real.exp_zero, Real.log_one,
      sub_zero, Real.log_exp]
    ring
  have eval_xMinusLogT : EMLTerm .eval x xMinusLogT = x - Real.log x := by
    simp only [hxml, EMLTerm .eval, eval_logT, Real.exp_log hx]
  have eval_logXMinusLogT :
      EMLTerm .eval x logXMinusLogT = Real.log (x - Real.log x) := by
    simp only [hlxml, EMLTerm .eval, eval_zeroT, eval_xMinusLogT,
      Real.exp_zero, Real.log_one, sub_zero, Real.log_exp]
    ring
  have eval_xMinus2LogT : EMLTerm .eval x xMinus2LogT = x - 2 * Real.log x := by
    simp only [hx2l, EMLTerm .eval, eval_logXMinusLogT, eval_logT,
      Real.exp_log (sub_log_pos hx), Real.log_one, sub_zero, Real.log_exp]
    ring
  have eval_twoLogT : EMLTerm .eval x twoLogT = 2 * Real.log x := by
    simp only [htl, EMLTerm .eval, eval_logT, eval_xMinus2LogT,
      Real.log_one, sub_zero]
    rw [Real.exp_log hx, Real.log_exp]
    ring
  show Real.exp (EMLTerm .eval x twoLogT) - Real.log (EMLTerm .eval x .one) = x ^ 2
  simp only [EMLTerm .eval, eval_twoLogT, Real.log_one, sub_zero]
```

```
  -- Goal: Real.exp (2 * Real.log x) = x ^ 2
  rw [show (2 :  ) * Real.log x = Real.log x + Real.log x from by ring,
      Real.exp_add, Real.exp_log hx, sq]

/-! ## Conjunct 9 (chunk 040): addition is EML-representable -/

private theorem c040_add_xy :
     t : EMLTerm ,   x y :  , EMLTerm .eval x y t = x + y := by
  refine  .eml
    (.eml .one (.eml (.eml .one (.eml .varX .one)) .one))
    (.eml
      (.eml (.eml .one (.eml (.eml .one (.eml .varX (.eml .varX .one))) .one))
            (.eml .varY .one))
      .one), ?_
  intro x y
  simp only [EMLTerm .eval, Real.log_one, sub_zero, Real.log_exp]
  have h1 : Real.exp 1 - (Real.exp 1 - x) = x := by ring
  have h2 : Real.exp 1 - (Real.exp 1 - Real.log (Real.exp x - x)) =
      Real.log (Real.exp x - x) := by ring
  rw [h1, h2, Real.exp_log (exp_sub_self_pos x)]
  ring

/-! ## Conjunct 10 (chunk 041): multiplication (positive case) is EML-representable -/

private theorem c041_mul_xy :
     t : EMLTerm ,   x y :  , 0 < x → 0 < y → EMLTerm .eval x y t = x * y := by
  refine  ?_, fun x y hx hy => ?_
  · exact .eml (.eml (.eml .one (.eml (.eml .one .varX) .one))
      (.eml (.eml (.eml .one (.eml (.eml .one
        (.eml (.eml .one (.eml (.eml .one .varX) .one))
          (.eml (.eml .one (.eml (.eml .one .varX) .one)) .one))) .one)) .varY) .one)) .one
  · simp only [EMLTerm .eval, Real.log_one, sub_zero, Real.log_exp]
    set e := Real.exp 1
    have h1 : e - (e - Real.log x) = Real.log x := by ring
    rw [h1]
    rw [Real.exp_log hx]
    have h3 : e - (e - Real.log (x - Real.log x)) = Real.log (x - Real.log x) := by ring
    rw [h3]
    rw [Real.exp_log (sub_log_pos hx)]
    have h5 : x - (x - Real.log x - Real.log y) = Real.log x + Real.log y := by ring
    rw [h5, Real.exp_add, Real.exp_log hx, Real.exp_log hy]

/-! ## Building blocks for Conjunct 11 (chunk 042) -/

private def pow_Z : EMLTerm  := .eml .one (.eml (.eml .one .one) .one)
private def pow_LOG (a : EMLTerm ) : EMLTerm  :=
  .eml pow_Z (.eml (.eml pow_Z a) .one)
private def pow_NEG_LOG (v raw : EMLTerm ) : EMLTerm  :=
  .eml (pow_LOG (.eml v raw)) (.eml raw .one)

private def pow_logx : EMLTerm  := pow_LOG .varX
private def pow_logy : EMLTerm  := pow_LOG .varY
private def pow_neg_logx : EMLTerm  := pow_NEG_LOG pow_logx .varX
private def pow_neg_logy : EMLTerm  := pow_NEG_LOG pow_logy .varY
private def pow_inv_y_plus_logy : EMLTerm  :=
  .eml pow_neg_logy (.eml pow_neg_logy .one)
```

```
private def pow_log_inv_y_plus_logy : EMLTerm := pow_LOG pow_inv_y_plus_logy
private def pow_inv_x_plus_logx : EMLTerm :=
  .eml pow_neg_logx (.eml pow_neg_logx .one)
private def pow_log_inv_x_plus_logx : EMLTerm := pow_LOG pow_inv_x_plus_logx
private def pow_A_arg : EMLTerm := .eml pow_log_inv_y_plus_logy
  (.eml (.eml pow_neg_logy (.eml pow_log_inv_x_plus_logx .one)) .one)
private def pow_B_arg : EMLTerm := .eml pow_log_inv_y_plus_logy
  (.eml (.eml pow_neg_logy (.eml pow_neg_logx .one)) .one)
private def pow_A : EMLTerm := .eml pow_A_arg .one
private def pow_B : EMLTerm := .eml pow_B_arg .one
private def pow_y_logx : EMLTerm := .eml (pow_LOG pow_A) (.eml pow_B .one)
private def pow_term : EMLTerm := .eml pow_y_logx .one

private lemma eval_pow_Z (x y : ) : EMLTerm .eval x y pow_Z = 0 := by
  simp [pow_Z, EMLTerm .eval, Real.log_one, Real.log_exp]

private lemma eval_pow_LOG (x y : ) (a : EMLTerm )
    (_ha : 0 < EMLTerm .eval x y a) :
    EMLTerm .eval x y (pow_LOG a) = Real.log (EMLTerm .eval x y a) := by
  simp only [pow_LOG, EMLTerm .eval, eval_pow_Z, Real.exp_zero, Real.log_one,
    sub_zero, Real.log_exp]
  ring

private lemma eval_pow_NEG_LOG (x y : ) (v raw : EMLTerm )
    (hraw : 0 < EMLTerm .eval x y raw)
    (hv : EMLTerm .eval x y v = Real.log (EMLTerm .eval x y raw)) :
    EMLTerm .eval x y (pow_NEG_LOG v raw) = -(EMLTerm .eval x y v) := by
  have h_inner_pos : 0 < EMLTerm .eval x y (.eml v raw) := by
    show 0 < Real.exp (EMLTerm .eval x y v) - Real.log (EMLTerm .eval x y raw)
    rw [hv]
    have : Real.log (EMLTerm .eval x y raw) + 1
        Real.exp (Real.log (EMLTerm .eval x y raw)) :=
      Real.add_one_le_exp _
    linarith
  show EMLTerm .eval x y (.eml (pow_LOG (.eml v raw)) (.eml raw .one)) = _
  simp only [EMLTerm .eval, Real.log_one, sub_zero]
  rw [eval_pow_LOG x y (.eml v raw) h_inner_pos]
  show Real.exp (Real.log (EMLTerm .eval x y (.eml v raw))) -
    Real.log (Real.exp (EMLTerm .eval x y raw)) = _
  rw [Real.log_exp, Real.exp_log h_inner_pos]
  show (Real.exp (EMLTerm .eval x y v) - Real.log (EMLTerm .eval x y raw)) -
    EMLTerm .eval x y raw = -(EMLTerm .eval x y v)
  rw [hv, Real.exp_log hraw]
  ring

private lemma eval_pow_term (x y : ) (hx : 0 < x) (hy : 0 < y) :
    EMLTerm .eval x y pow_term = Real.exp (y * Real.log x) := by
  -- evaluations of building blocks
  have h_var_x : EMLTerm .eval x y .varX = x := rfl
  have h_var_y : EMLTerm .eval x y .varY = y := rfl
  have h_logx : EMLTerm .eval x y pow_logx = Real.log x := by
    show EMLTerm .eval x y (pow_LOG .varX) = Real.log x
    rw [eval_pow_LOG x y .varX (h_var_x  hx), h_var_x]
  have h_logy : EMLTerm .eval x y pow_logy = Real.log y := by
    show EMLTerm .eval x y (pow_LOG .varY) = Real.log y
    rw [eval_pow_LOG x y .varY (h_var_y  hy), h_var_y]
```

```
have h_neg_logx : EMLTerm.eval x y pow_neg_logx = -Real.log x := by
  have : EMLTerm.eval x y pow_neg_logx = -EMLTerm.eval x y pow_logx := by
    simp only [pow_neg_logx]
    exact eval_pow_NEG_LOG x y pow_logx .varX hx h_logx
  rw [this, h_logx]
have h_neg_logy : EMLTerm.eval x y pow_neg_logy = -Real.log y := by
  have : EMLTerm.eval x y pow_neg_logy = -EMLTerm.eval x y pow_logy := by
    simp only [pow_neg_logy]
    exact eval_pow_NEG_LOG x y pow_logy .varY hy h_logy
  rw [this, h_logy]
have h_inv_y_plus_logy :
    EMLTerm.eval x y pow_inv_y_plus_logy = y⁻¹ + Real.log y := by
  show Real.exp (EMLTerm.eval x y pow_neg_logy) -
      Real.log (Real.exp (EMLTerm.eval x y pow_neg_logy) - Real.log 1) = _
  rw [Real.log_one, sub_zero, Real.log_exp, h_neg_logy, Real.exp_neg, Real.exp_log hy]
  ring
have h_inv_x_plus_logx :
    EMLTerm.eval x y pow_inv_x_plus_logx = x⁻¹ + Real.log x := by
  show Real.exp (EMLTerm.eval x y pow_neg_logx) -
      Real.log (Real.exp (EMLTerm.eval x y pow_neg_logx) - Real.log 1) = _
  rw [Real.log_one, sub_zero, Real.log_exp, h_neg_logx, Real.exp_neg, Real.exp_log hx]
  ring
have h_inv_y_pos : 0 < EMLTerm.eval x y pow_inv_y_plus_logy := by
  rw [h_inv_y_plus_logy]; exact inv_add_log_pos hy
have h_inv_x_pos : 0 < EMLTerm.eval x y pow_inv_x_plus_logx := by
  rw [h_inv_x_plus_logx]; exact inv_add_log_pos hx
have h_log_inv_y :
    EMLTerm.eval x y pow_log_inv_y_plus_logy =
      Real.log (y⁻¹ + Real.log y) := by
  simp only [pow_log_inv_y_plus_logy]
  rw [eval_pow_LOG x y pow_inv_y_plus_logy h_inv_y_pos, h_inv_y_plus_logy]
have h_log_inv_x :
    EMLTerm.eval x y pow_log_inv_x_plus_logx =
      Real.log (x⁻¹ + Real.log x) := by
  simp only [pow_log_inv_x_plus_logx]
  rw [eval_pow_LOG x y pow_inv_x_plus_logx h_inv_x_pos, h_inv_x_plus_logx]
-- inner_y_x := .eml pow_neg_logy (.eml pow_log_inv_x_plus_logx .one)
--   = exp(-log y) - log(exp(log(x⁻¹ + log x)) - log 1)
--   = 1/y - log(x⁻¹ + log x)
have h_xinv_logx_pos : 0 < x⁻¹ + Real.log x := inv_add_log_pos hx
have h_inner_y_x :
    EMLTerm.eval x y (.eml pow_neg_logy (.eml pow_log_inv_x_plus_logx .one)) =
      y⁻¹ - Real.log (x⁻¹ + Real.log x) := by
  show Real.exp (EMLTerm.eval x y pow_neg_logy) -
    Real.log (Real.exp (EMLTerm.eval x y pow_log_inv_x_plus_logx) -
      Real.log 1) = _
  rw [Real.log_one, sub_zero, Real.log_exp, h_log_inv_x, h_neg_logy,
      Real.exp_neg, Real.exp_log hy]
have h_A_arg :
    EMLTerm.eval x y pow_A_arg = Real.log y + Real.log (x⁻¹ + Real.log x) := by
  show Real.exp (EMLTerm.eval x y pow_log_inv_y_plus_logy) -
    Real.log (Real.exp
      (EMLTerm.eval x y (.eml pow_neg_logy (.eml pow_log_inv_x_plus_logx .one)))
      - Real.log 1) = _
  rw [Real.log_one, sub_zero, Real.log_exp, h_log_inv_y,
      Real.exp_log (inv_add_log_pos hy), h_inner_y_x]
```

```
    ring
  have h_A : EMLTerm.eval x y pow_A = y * (x⁻¹ + Real.log x) := by
    show Real.exp (EMLTerm.eval x y pow_A_arg) - Real.log 1 = _
    rw [Real.log_one, sub_zero, h_A_arg, Real.exp_add,
        Real.exp_log hy, Real.exp_log h_xinv_logx_pos]
  have h_inner_y_x_2 :
      EMLTerm.eval x y (.eml pow_neg_logy (.eml pow_neg_logx .one)) =
        y⁻¹ + Real.log x := by
    show Real.exp (EMLTerm.eval x y pow_neg_logy) -
      Real.log (Real.exp (EMLTerm.eval x y pow_neg_logx) - Real.log 1) = _
    rw [Real.log_one, sub_zero, Real.log_exp, h_neg_logx, h_neg_logy,
        Real.exp_neg, Real.exp_log hy]
    ring
  have h_B_arg : EMLTerm.eval x y pow_B_arg = Real.log y - Real.log x := by
    show Real.exp (EMLTerm.eval x y pow_log_inv_y_plus_logy) -
      Real.log (Real.exp
        (EMLTerm.eval x y (.eml pow_neg_logy (.eml pow_neg_logx .one)))
        - Real.log 1) = _
    rw [Real.log_one, sub_zero, Real.log_exp, h_log_inv_y,
        Real.exp_log (inv_add_log_pos hy), h_inner_y_x_2]
    ring
  have h_B : EMLTerm.eval x y pow_B = y / x := by
    show Real.exp (EMLTerm.eval x y pow_B_arg) - Real.log 1 = _
    rw [Real.log_one, sub_zero, h_B_arg, Real.exp_sub,
        Real.exp_log hy, Real.exp_log hx]
  have h_A_pos : 0 < EMLTerm.eval x y pow_A := by
    rw [h_A]; exact mul_pos hy h_xinv_logx_pos
  have h_log_A : EMLTerm.eval x y (pow_LOG pow_A) =
      Real.log (y * (x⁻¹ + Real.log x)) := by
    rw [eval_pow_LOG x y pow_A h_A_pos, h_A]
  have h_y_logx : EMLTerm.eval x y pow_y_logx = y * Real.log x := by
    show Real.exp (EMLTerm.eval x y (pow_LOG pow_A)) -
      Real.log (Real.exp (EMLTerm.eval x y pow_B) - Real.log 1) = _
    rw [Real.log_one, sub_zero, Real.log_exp, h_log_A, h_B,
        Real.exp_log (mul_pos hy h_xinv_logx_pos)]
    field_simp
    ring
  show Real.exp (EMLTerm.eval x y pow_y_logx) - Real.log 1 = _
  rw [Real.log_one, sub_zero, h_y_logx]

/-! ## Conjunct 11 (chunk 042): real power (positive case) is EML-representable -/

private theorem c042_pow_xy :
      t : EMLTerm ,   x y :  , 0 < x → 0 < y → EMLTerm.eval x y t = x ^ y := by
  refine  pow_term, fun x y hx hy => ?_
  rw [eval_pow_term x y hx hy, Real.rpow_def_of_pos hx]
  ring_nf

/-! ## Umbrella theorem -/

/-- Main completeness umbrella: each of the eleven constructive sub-cases
of the EML decomposition has a witnessing term whose evaluation realises
the target value or function. Conjuncts in order:

1. zero (chunk 030)
2. -1 (chunk 031)
```

```
3. 2 (chunk 032)
4. 1/2 (chunk 033)
5. e (chunk 022)
6. negation, x  -x (chunk 036)
7. reciprocal on positives, x  1/x (chunk 037)
8. square on positives, x  x² (chunk 038)
9. addition, (x,y)  x+y (chunk 040)
10. multiplication on positive quadrant, (x,y)  x·y (chunk 041)
11. real power on positive quadrant, (x,y)  x^y (chunk 042)

NOT included:  (chunk 034), i (chunk 035), √x (chunk 039) - their
constructions require the paper's Supplementary trees and remain
permanent sorries. -/
theorem main_completeness :
    ( t : EMLTerm, EMLTerm.eval t = 0)
    ( t : EMLTerm, EMLTerm.eval t = -1)
    ( t : EMLTerm, EMLTerm.eval t = 2)
    ( t : EMLTerm, EMLTerm.eval t = 1 / 2)
    ( t : EMLTerm, EMLTerm.eval t = Real.exp 1)
    ( t : EMLTerm ,  x :  , EMLTerm .eval x t = -x)
    ( t : EMLTerm ,  x :  , 0 < x → EMLTerm .eval x t = 1 / x)
    ( t : EMLTerm ,  x :  , 0 < x → EMLTerm .eval x t = x ^ 2)
    ( t : EMLTerm ,  x y :  , EMLTerm .eval x y t = x + y)
    ( t : EMLTerm ,  x y :  , 0 < x → 0 < y → EMLTerm .eval x y t = x * y)
    ( t : EMLTerm ,  x y :  , 0 < x → 0 < y → EMLTerm .eval x y t = x ^ y) :=
   c030_zero, c031_neg_one, c032_two, c033_half, c022_e,
    c036_neg_x, c037_inv_x, c038_sq_x,
    c040_add_xy, c041_mul_xy, c042_pow_xy

end EML
```